Deep Reinforcement Learning for the Interval Job Shop Scheduling Problem: A Comparison with Genetic Programming Hyper-Heuristics across Inference Budgets

Hernán Díaz Rodríguez[a]

[a]*Department of Computing, University of Oviedo, Gijón, Spain*

Abstract

The interval job shop scheduling problem (IJSP) models each uncertain processing time as an interval between known bounds and is solved in the literature by per-instance metaheuristic search. This paper presents a constructive deep reinforcement learning policy for the problem. Its state features are dimensionless and its encoder is permutation-invariant, so one trained network applies to instances of any size, with no retraining and no change of architecture. Trained on instances of a single size class, the validation-selected policy improves on every hand-crafted dispatching rule on all seventy benchmark instances, six of the seven size classes unseen. It is then compared against a dispatching rule evolved by genetic programming for the same benchmark, thirty trained artifacts per family evaluated in one harness at matched inference budgets. Which of the two leads is decided by the budget, and they differ significantly at every budget evaluated: the rule leads by 0.8 points of relative error given one constructive pass each, and the policy leads by the same margin once both draw samples; on twelve classical instances outside the training family the policy leads at every budget. The paradigms also differ in where an objective that values robustness acts: retrained under a width-penalizing objective, the policy narrows its schedules through sample selection, its weights unmoved, whereas the same objective acts through the evolved representation in the rules.

Keywords: job shop scheduling, interval uncertainty, deep reinforcement learning, neural combinatorial optimization, genetic programming hyper-heuristics, size invariance

1. Introduction

In the job shop scheduling problem (JSP), every job visits the machines of a shop in its own prescribed order, and the task is to sequence the operations on each machine so that the overall completion time is as small as possible. The problem lies at the core of manufacturing operations planning: NP-hard [14] and industrially ubiquitous, it is studied predominantly under the assumption that every processing time is known exactly. Real production environments seldom satisfy that assumption: schedules are committed before the work they plan is executed, and the processing times they assume are no more than estimates, displaced in practice by tool wear, operator variability and material differences.

Email address: diazhernan@uniovi.es (Hernán Díaz Rodríguez)

Among the formalisms proposed for this uncertainty, stochastic and fuzzy models [16] require the decision maker to supply probability distributions or membership functions; the *interval* model requires only bounds. In the resulting interval JSP (IJSP), each duration is a closed interval, schedule evaluation propagates those intervals, and the makespan is itself an interval, compared through a ranking criterion [25, 9].

The state of the art in IJSP makespan minimization is currently held by per-instance metaheuristic search: elitist artificial bee colony algorithms [10, 9] reach mean relative errors (RE, measured against crisp Taillard lower bounds) between roughly 6% and 16% depending on the size class. This cost is structural: the search must be repeated for every new instance, at seconds to minutes per run on dedicated hardware.

At the opposite end of the computational spectrum lie *constructive* methods, which build a schedule in a single pass without search. The classical representatives are priority dispatching rules, myopic functions such as shortest processing time or most operations remaining [32, 18], which are fast and interpretable but leave a wide quality gap on the IJSP. Genetic programming (GP) hyper-heuristics narrow that gap by evolving dispatching rules offline from a training set [4, 29]; a rule recently evolved for the interval benchmark studied here [12] constitutes the strongest constructive baseline available. Both families deliver schedules at negligible per-instance cost, which makes them the natural referents and competitors for a learned policy.

The deterministic scheduling literature has developed a third constructive approach, *learning to dispatch*: a deep reinforcement learning (DRL) agent builds the schedule constructively, choosing one eligible operation at a time [46, 33, 39]. The training cost is incurred once; thereafter the policy returns a schedule for a new instance in seconds, which makes it attractive both as a standalone solver under tight time budgets and as an initializer for search. This paradigm has not, however, been extended to interval durations: a recent survey of learning-based job shop scheduling [43] lists no method addressing interval-valued processing times.

Two obstacles separate standard JSP-DRL designs from the IJSP. The first is representational: the natural assumption, and the one this line of work has operated under, is that the uncertainty must be observable to the agent, which should treat an operation with duration $[10, 11]$ differently from one with duration $[5, 16]$ even though both share the same midpoint. The second is architectural: the amortization argument that motivates learning requires one network to serve instances of any size. Designs that encode the instance as a fixed-dimension tensor are tied to the size class they were trained on, and graph-based encoders achieve size transfer [46, 33] through message passing over the full disjunctive graph; whether the same invariance can be obtained with a substantially simpler encoder is a design question this paper answers in the affirmative.

This paper builds the policy those obstacles call for and uses it to compare the two families of learned constructive heuristic on the same ground. *Q1:* which of the two produces the better schedules for the IJSP, and does the answer hold across inference budgets, instance sizes and shop families? *Q2:* where in each paradigm does the handling of the uncertainty live, that is, which uses of the interval information are load-bearing for a policy and which are inert, and at what stage of each method does an objective that values robustness act? *Q3:* what does either learner buy over the hand-crafted rules it replaces, and where do both stand relative to the per-instance metaheuristic state of the art?

Contributions..

1. *A size-invariant constructive DRL policy for the IJSP* (Section 4), and to the best of our knowledge the first application of deep reinforcement learning to this problem: the uncertainty axis of the neural scheduling literature reaches stochastic and fuzzy durations but not interval ones (Section 2.4). All features are dimensionless: temporal quantities are scaled by a per-instance lower bound and interval uncertainty enters as relative widths. A Deep Sets [44] encoder over the eligible-operation set, combined with a pooled global context, yields policy *and* value networks with no dimension depending on the numbers of jobs and machines.
2. *A head-to-head comparison of the two learned constructive paradigms, deep reinforcement learning and genetic programming hyper-heuristics, under a shared protocol* (Sections 6.2 and 6.3), whose absence from the literature the survey of Xu et al. [43] highlights. Thirty evolved rules against thirty trained policies, on the 70 benchmark instances and on twelve classical ones, run in a single harness that fixes the instances, the interval arithmetic, the ranking criterion, the inference budgets and the execution realizations, with per-instance results reported in full (supplementary material). Which of the two leads is decided by the budget, and they differ significantly at every budget evaluated.
3. *A localization of where each paradigm handles the uncertainty* (Section 7.3), in three parts. The mechanism the design offers: schedule selection ranks by the worst case first and consults widths only on ties of that key, measured to decide two per cent of selections, while the training signal retains one lower-endpoint term, so what the widths contribute is settled empirically rather than assumed. Its examination on the trained policy: a permutation audit and two retraining ablations that probe the widths as inputs, whose effect on unseen instances is bounded within a third of a point either way, and the intervals as training data. And the contrast between the paradigms: a width-penalizing objective acts through sample selection in the policy, with the weights unmoved, and through the evolved representation in the GP study [12].
4. *An account of which standard components earn their cost* (Sections 4.3, 5.3 and 7): best-of-N inference does, self-attention and three automatic-configuration campaigns do not, and under Monte Carlo execution the learned schedules track their predicted makespans more faithfully than those of every hand-crafted rule.

Section 2 reviews related work and Section 3 states the problem. Section 4 presents the policy, Section 5 the experimental methodology and Section 6 the results; Section 7 analyses them and states the limitations, and Section 8 concludes.

2. Related Work

2.1. Job shop scheduling under interval uncertainty

Uncertain processing times have been modeled with probability distributions, fuzzy numbers and intervals. The fuzzy line is the most mature: durations are represented as fuzzy numbers, schedules are compared through the expected value of their fuzzy makespan, and two decades of population-based solvers have consolidated both the model and its evaluation conventions [16, 1, 31]. The interval model can be read as the minimal-commitment

limit of that representation: it retains only the support bounds, asks the decision maker for no shape information inside them, and is the standard minimal description of uncertainty in robust discrete optimization at large, where it underpins min–max and min–max-regret formulations [23, 20]. Lei [25] opened the interval JSP line, propagating interval durations through the schedule with interval arithmetic and steering a population-based neighborhood search by direct comparison of interval makespans. Later work broadened both the model and its toolkit. Díaz et al. [7] compared the candidate interval rankings systematically within a genetic algorithm and identified the lexicographic criterion adopted in this paper as the most robust choice for the problem; Díaz et al. [8] shifted the objective to total tardiness and paired it with explicit robustness measures; Sotskov et al. [37] delimited analytically solvable special cases; and Zheng et al. [47] carried interval durations into richer shop variants. On the Taillard-derived benchmark used here, the strongest published makespan results belong to the elitist artificial bee colony family [9, 10], whose successive refinements, elitist replacement and neighborhood filtering among them, progressively reduced the mean relative error to the single digits on most size classes; these works also fixed the evaluation conventions this paper adopts (interval arithmetic, lexicographic ranking during construction, expected makespan for reporting). Common to the whole line is per-instance search; fast constructive alternatives for the IJSP have begun to be explored only recently, through the evolved rule discussed in Section 2.3.

2.2. Priority dispatching rules

The oldest constructive approach to the job shop builds a schedule in a single left-to-right pass, repeatedly choosing which of the eligible operations to place next. A *priority dispatching rule* makes that choice by scoring the candidates with a fixed formula over their attributes. Surveys catalogue well over a hundred variants [32, 18, 40]; the baselines of this paper are the classical single-attribute ones, retained with their usual acronyms. Shortest and longest processing time (SPT, LPT) read the operation itself, earliest starting time (EST) reads the schedule built so far, and most work remaining and most operations remaining (MWKR, MOR) read the job the operation belongs to, counting its pending work in time units and in operations respectively. Under interval durations these attributes are intervals, and candidates are compared with the ranking of Section 3.

The active-schedule generator of Giffler and Thompson [15] is a refinement of the dispatching step rather than a rule: it restricts the candidates to the conflict set of the machine that would finish earliest and applies a priority rule only within that set, guaranteeing *active* schedules, in which no operation can be started earlier without delaying another. The restriction is worth more than the choice of rule it wraps, taking MWKR from 64.7% to 29.5% of RE on the benchmark used here; G&T-SPT and G&T-MWKR denote the two combinations reported below.

One pass and no search makes these rules the cheapest constructive option available, and a fixed formula suiting neither every shop nor every objective [18] is what motivates learning one instead, by either of the two routes that follow. It is also why a constructive policy is measured against them rather than against per-instance metaheuristics, which belong to a different computational class.

2.3. Genetic programming hyper-heuristics

Learning constructive heuristics is older than its neural incarnation. Genetic programming hyper-heuristics evolve priority expressions, trees over hand-crafted attributes such as processing time or work remaining, offline on a training set, and deploy the best expression as an ordinary dispatching rule: the artifact is interpretable and its inference cost is that of evaluating a formula. Design choices (attribute sets, fitness, generalization across instances) are reviewed by Branke et al. [4], and Nguyen et al. [29] organize the broader family under a unified framework. For the job shop specifically, Nguyen et al. [30] showed that the choice of rule representation materially shapes the quality of the evolved dispatchers, and Karunakaran et al. [19] evolved rules for dynamic job shops with uncertain processing times, exposing duration deviations to the rules as attributes, under an uncertainty model that is, once again, distributional rather than interval-valued. A recent study [12] develops the GP side for the interval benchmark used here, evolving interval-aware rules under the same ranking and evaluation conventions, with both a deterministic pass and a sampling variant.

2.4. Neural constructive policies for scheduling

Neural combinatorial optimization began with pointer networks [41], sequence models that emit a permutation of their input and can therefore represent tours and schedules; Bello et al. [2] trained them with policy gradients, removing the need for supervised targets, and attention-based encoder–decoders trained against greedy-rollout baselines then became the standard for routing [22, 28]. Reinforcement learning for job-shop dispatching predates the deep era: Gabel and Riedmiller [13] already learned decentralized dispatching policies by gradient ascent. Its modern applications to machine scheduling are surveyed by Kayhan and Yildiz [21]. For the deterministic JSP the reference design is that of Zhang et al. [46]: the partial schedule is encoded as a disjunctive graph, a graph neural network embeds it, and the policy trained by PPO on small synthetic instances generalizes zero-shot to larger ones while surpassing the classical priority rules. Park et al. [33] refine that representation and document the cross-size generalization explicitly, and Tassel et al. [39] show that the environment formulation itself (state design and reward shaping) accounts for a substantial share of the attainable performance. The paradigm has since expanded along three axes: to richer shops, with heterogeneous-graph policies for the flexible JSP [36]; from construction to improvement, with DRL-guided local search over complete schedules [45]; and toward uncertainty. On that third axis, Grumbach et al. [17] size slack buffers in stochastic flow shops under a distributional, event-driven model; Smit et al. [35] extend constructive policies to stochastic processing times by processing sampled duration scenarios with an attention module; and Wang et al. [42] train an autoregressive generative model for the fuzzy job shop. The interval representation, the minimal of these uncertainty models, remains unaddressed by this literature, which is the gap the present paper fills.

Two further components of the toolbox matter here. Sampling several rollouts at inference and keeping the best is a standard quality lever whose returns grow with the diversity of the learned distribution [22, 24]. Permutation-invariant set encoders were formalized as Deep Sets [44], of which self-attention is the natural strict generalization; the architecture of Section 4.3 builds on the former, and the latter is evaluated as a controlled ablation in the supplementary material.

The survey by Xu et al. [43] compares the GP and RL families for job shop scheduling and notes the scarcity of comparisons under a common protocol. The availability of the evolved rule of Section 2.3 on the very benchmark studied here lets Section 6.2 put the two learned artifacts on the same instances at matched *inference* budgets.

3. Problem Statement

Definition 1 (IJSP)**.** An IJSP instance comprises n jobs $J_1, \ldots, J_n$ and m machines, indexed $1, \ldots, m$. Job J_j is a fixed sequence of m operations $(o_{j1}, \ldots, o_{jm})$, where operation o_{jk} requires machine $\nu_{jk} \in \{1, \ldots, m\}$ and an uncertain processing time known only to lie in the interval $\mathbf{p}_{jk} = [p^L_{jk}, p^U_{jk}]$, with $0 \le p^L_{jk} \le p^U_{jk}$. Each job visits each machine exactly once, so $(\nu_{j1}, \ldots, \nu_{jm})$ is a permutation of the machine indices. Each machine processes at most one operation at a time, operations of the same job cannot overlap, and preemption is not allowed.

A solution is a processing order of all nm operations, that is, a sequence in which o_{jk} appears before $o_{j,k+1}$ for every job j and every $k < m$. We encode it as a *permutation with repetition* of job indices, where the k-th occurrence of j denotes o_{jk}. The order induces, on each machine, a sequence over the operations that require it, and its *semiactive schedule* starts every operation as early as those sequences permit: no operation can start before its job predecessor $JP(o_{jk}) = o_{j,k-1}$ or its machine predecessor $MP(o_{jk})$, the operation immediately preceding o_{jk} on machine ν_{jk} in the induced sequence, have completed. With component-wise interval addition $[a, b]+[c, d] = [a+c,\ b+d]$ and join $\max([a, b], [c, d]) = [\max(a, c), \max(b, d)]$, the start and completion times of every operation are themselves intervals, given by the recursion

$$\mathbf{s}_o = \max\Big(\mathbf{c}_{JP(o)},\ \mathbf{c}_{MP(o)}\Big), \qquad \mathbf{c}_o = \mathbf{s}_o + \mathbf{p}_o, \tag{1}$$

When a predecessor does not exist (the job predecessor of a job's first operation, or the machine predecessor of the first operation scheduled on a machine), its completion is taken as $[0, 0]$, so that the corresponding argument is neutral in the max and the start is decided by the predecessor that does exist. The interval makespan collects the job completions component-wise,

$$\mathbf{C}_{\max} = \Big[\max_j c^L_{o_{jm}},\ \max_j c^U_{o_{jm}}\Big] =: \Big[C^L_{\max},\ C^U_{\max}\Big]. \tag{2}$$

A *realization* of the instance assigns to each operation a crisp duration $p_{jk} \in \mathbf{p}_{jk}$; *executing* a processing order under a realization means evaluating the recursion of Eq. (1) with those crisp durations, which yields a crisp makespan. Because the recursion is monotone in the durations, the executed makespan of *every* realization falls inside $\mathbf{C}_{\max}$. Intervals carry no canonical total order, so schedule comparison requires a ranking choice; during construction and search we use the lexicographic order

$$\mathbf{a} \prec_{lex} \mathbf{b} \iff a^U < b^U \ \vee\ \Big(a^U = b^U \wedge a^L < b^L\Big) \tag{3}$$

for intervals $\mathbf{a} = [a^L, a^U]$ and $\mathbf{b} = [b^L, b^U]$, which prioritizes the worst case, in line with min–max robust optimization [3, 20]. The optimization problem is then to find a processing order whose interval makespan is minimal under this order. This modeling choice is not reopened

here: a systematic study of the alternative ranking criteria found the lexicographic order the most robust for this problem [7], and it is kept fixed throughout so that every method in this paper is judged by the same criterion. For reporting we follow the field convention [7, 9], which writes the interval midpoint as $E[\mathbf{C}_{\max}]$ and uses it as the scalar summary of an interval makespan; the notation is conventional, as under an interval-only model of uncertainty no distribution is specified and the midpoint is a proxy rather than a derived expectation:

$$\mathrm{RE}(\%) = \frac{E[\mathbf{C}_{\max}] - \mathrm{LB}}{\mathrm{LB}} \times 100, \qquad E[\mathbf{C}_{\max}] = \tfrac{1}{2}\Big(C^{L}_{\max} + C^{U}_{\max}\Big), \tag{4}$$

with LB the published lower bound of the crisp Taillard counterpart [38], an indicative, hardware-independent yardstick rather than a strict optimality gap, since the crisp JSP and the IJSP are different problems.

Every element of this section is shared with the genetic programming study this paper compares against [12].

4. Method

Reinforcement learning is learning to decide from experience. An *agent* interacts with an environment in steps: it observes the current *state* s, chooses an *action* a among those available, and receives a numeric *reward* signalling how good the consequences were. The agent's behavior is its *policy* $\pi(a \mid s)$: a function that, given the state s, assigns a probability to each available action a. The policy is the object of learning: training increases the probability of actions followed by a high cumulative reward (the *return*).

Policy-gradient methods such as PPO [34] judge an action not by its return alone but by comparing that return with what was *expected* from the state where the action was taken; only the surplus, the *advantage*, moves the policy. The expectation is itself learned: a *value function* $V(s)$ estimates the return attainable from state s. In practice both functions share one network with two outputs, a *policy head* that scores the available actions and a *value head* that estimates $V(s)$, trained jointly. This is why the network in this paper has two heads.

Instantiated on scheduling, the agent is a dispatcher and schedule construction is a finite-horizon *Markov decision process* of exactly nm steps, one per operation. The state after t steps is the partial schedule (each job's next-operation pointer, the job and machine completion intervals), a sufficient statistic for the remaining decisions; the available actions are the *eligible* operations, that is, the next unscheduled operation of each job, collected in the set $\mathcal{E}_t$ with $1 \le |\mathcal{E}_t| \le n$ and written $\mathcal{E}$ when the step is immaterial. The generic $\pi(a \mid s)$ thus specializes to $\pi(i \mid s)$, a distribution over which eligible operation i to schedule next; the transition applies Eq. (1) deterministically; an *episode* is the construction of one complete schedule, and the reward is a function of the makespan achieved. The trained policy is then a dispatching criterion in its own right: at each step it orders the eligible operations by preferences learned from experience rather than by a fixed, hand-written formula.

Figure 1 shows the network, read from the input at the top down to the two outputs. Each eligible operation enters as a row of 16 numbers that describe it: how long it may take, how much work its job still carries, how soon it could start (Section 4.2). A multilayer

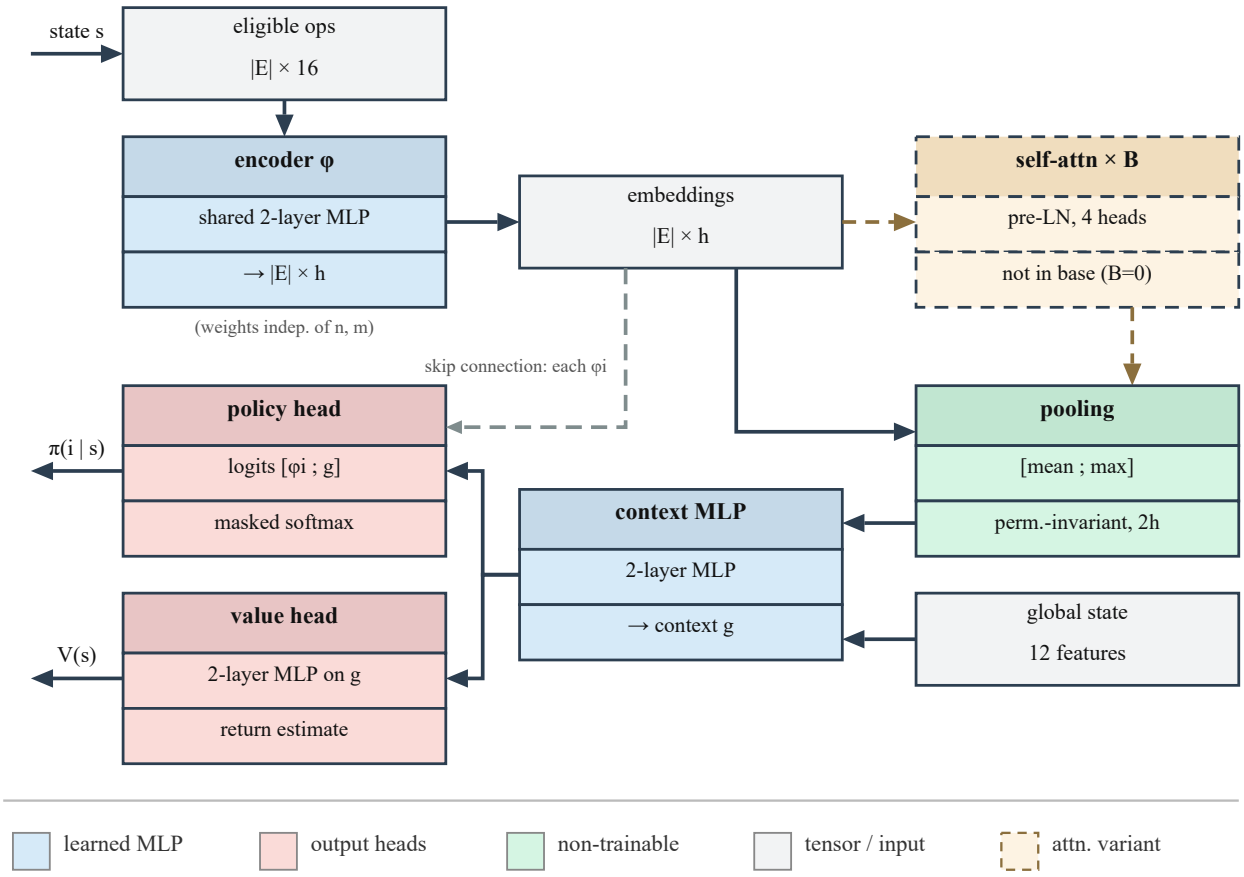

Figure 1: The size-invariant policy/value network. Solid edges are the base model's data path; the grey dashed edge is the per-candidate skip connection; the dashed detour is the self-attention variant reported in the supplementary material, not part of the base model (B=0).

perceptron (MLP), the *same* one for every candidate, turns each row into an *embedding*: a vector of h learned quantities that re-describe the candidate.

The number of candidates varies from step to step, so the embeddings are *pooled*: their mean and their componentwise maximum are two fixed-size summaries of the whole set. Joined with 12 numbers describing the shop as a whole (the *global state*), these summaries are condensed by another MLP into a single *context* vector g, a fixed-size summary of the current decision state.

The two heads introduced above read from this context. Pooling, however, discards by design which candidate is which, so the context alone cannot rank them. The policy head therefore receives two inputs, the shared context g and each candidate's own embedding, carried past the pooling stage by the *skip connection* visible in Figure 1. The head is applied once per candidate: for eligible operation i it reads the pair (embedding of i, context) and outputs a single scalar score, so $|\mathcal{E}|$ candidates yield $|\mathcal{E}|$ scores, each tied to its operation by construction because it was computed from that operation's embedding. A *softmax* over these scores (the *logits*, in network terms) turns them into the probabilities $\pi(i \mid s)$, one per eligible operation, however many there are (*masked* means that unused padding slots are

excluded). The value head estimates the situation rather than any one candidate, so it reads g alone and outputs $V(s)$. In total, four two-layer MLPs (the candidate encoder ϕ, the context combiner, and the two heads) are composed into a single network and trained jointly end to end.

The dashed detour between the embeddings and the pooling is an *optional* self-attention stage, evaluated as a variant in the supplementary material; the base model, which every headline result of this paper uses, omits it.

Sections 4.1–4.3 specify, in order, the reward, the 16+12 input quantities, and the network stages. Training and inference are described with the experimental setting in Section 5.4.

4.1. The reward

The *reward* at step t is a weighted sum of six shaping components,

$$r_t \;=\; w_{\mathrm{mk}}\, r_t^{\mathrm{mk}} + w_{\mathrm{pr}}\, r_t^{\mathrm{pr}} + w_{\mathrm{id}}\, r_t^{\mathrm{id}} + w_{\mathrm{li}}\, r_t^{\mathrm{li}} + w_{\mathrm{cr}}\, r_t^{\mathrm{cr}} + w_{\mathrm{ba}}\, r_t^{\mathrm{ba}}, \tag{5}$$

where each $r_t^{\bullet}$ is one shaping component and $w_{\bullet} \in [0,1]$ its fixed weight: terminal makespan (mk), job progress (pr), machine idleness (id), local improvement (li), operation criticality (cr) and load balance (ba), each defined below.

Every component is dimensionless, and comparable across instances and sizes: five are divided by a magnitude of the instance, listed with them, and job progress is by construction a fraction of the episode. Several of those scales use $\widehat{\mathrm{LB}}$, a lower bound on the makespan that the method computes from the instance itself as the larger of the two classical relaxations, the most loaded machine and the longest job, in worst-case durations:

$$\widehat{\mathrm{LB}} = \max\Big(\max_{\mu} \sum_{(j,k)\,:\,\nu_{jk}=\mu} p_{jk}^{U},\;\; \max_{j} \sum_{k=1}^{m} p_{jk}^{U} \Big).$$

It is an internal estimate, so the method does not depend on externally supplied references. In the definitions below, write c_{μ}^{U} for the worst-case completion time of machine μ over the operations scheduled so far, $M_t^{U} = \max_{\mu} c_{\mu}^{U}$ for the projected worst-case makespan after t decisions, and let step t schedule operation o_{jk} on machine $\mu = \nu_{jk}$. The six components are:

- *Terminal makespan* (w_{mk}=1.0): zero at every step except the last, where

$$r_{nm}^{\mathrm{mk}} = -\frac{C_{\max}^{U}}{s_{\mathrm{mk}}}, \qquad s_{\mathrm{mk}} = \frac{\widehat{\mathrm{LB}}}{10},$$

  Substituting the scale gives $r_{nm}^{\mathrm{mk}} = -10\,(1+g)$, with $g = (C_{\max}^{U} - \widehat{\mathrm{LB}})/\widehat{\mathrm{LB}}$ the relative gap to the instance bound: a function of the relative gap alone, so its magnitude is independent of the instance's size and time scale, and a schedule of the same relative quality receives the same reward on any instance. The robust arms of Section 7.3 replace $C_{\max}^{U}$ here by f_{λ}.

- *Job progress* (w_{pr}=0.05):

$$r_t^{\mathrm{pr}} = \frac{1}{nm}.$$

A constant per operation scheduled, on every step but the terminal one, where the terminal component replaces it; the episode sum is $(nm-1)/nm$. With $\gamma = 1$ and the fixed nm-step horizon this term is the same for every complete schedule, so it stabilizes value estimation rather than shaping the policy's preferences.

- *Machine idleness* (w_{id}=0.15): when the job predecessor decides the start of o_{jk} under the ranking of Eq. (3),

$$r_t^{\mathrm{id}} = -\frac{s^U_{o_{jk}} - c^L_\mu}{s_{\mathrm{id}}}, \qquad s_{\mathrm{id}} = 2\bar{p},$$

and zero otherwise, with $\bar{p}$ the mean midpoint duration of the instance: the upper endpoint of the idle interval that scheduling o_{jk} opens on its machine, measured in units of typical operations. An idle gap is widest when the machine freed at its earliest, so the worst case of an idle interval reads the machine's *lower* completion endpoint; this is the one place where the training signal consults a lower endpoint, a fact Section 7.3 returns to.

- *Local improvement* (w_{li}=0.30):

$$r_t^{\mathrm{li}} = \kappa_t \, \frac{M^U_{t-1} - M^U_t}{s_{\mathrm{mk}}/10}, \qquad \kappa_t = \begin{cases} 2 & M^U_t > M^U_{t-1}, \\ 1 & \text{otherwise}, \end{cases}$$

the step change of the projected makespan, with deteriorations penalized twice as strongly: a dense signal so that early decisions are not credited only nm steps later. On the first step of an episode the component is zero, as no previous projection exists.

- *Operation criticality* (w_{cr}=0.05):

$$r_t^{\mathrm{cr}} = \frac{\sum_{k'>k} p^U_{jk'}}{s_{\mathrm{cr}}}, \qquad s_{\mathrm{cr}} = \frac{W^U}{2n},$$

with W^U the instance's total worst-case work: the worst-case work remaining in the scheduled operation's job. Favoring jobs with much work remaining is a standard proxy for the emerging critical path.

- *Load balance* (w_{ba}=0.10; see below):

$$r_t^{\mathrm{ba}} = -\frac{\sigma_\mu\left(c^U_\mu\right)}{s_{\mathrm{ba}}}, \qquad s_{\mathrm{ba}} = 1.5\,\sigma_\mu\left(L^U_\mu\right),$$

where σ_μ denotes the standard deviation across the machines already in use (machines with no operation scheduled yet are excluded), taken after the step over the worst-case completion times and, in the scale, over the total worst-case loads $L^U_\mu = \sum_{(j,k)\,:\,\nu_{jk}=\mu} p^U_{jk}$ of the instance. Referring the observed dispersion to the dispersion the instance's own loads exhibit discourages lopsided machine loads without penalizing an instance for being intrinsically unbalanced.

The training entry point sets the six weights as fixed constants, with w_{ba} nominally 0.15. A per-instance adjustment layer can then override three of them by fixed deterministic threshold rules of the instance's own statistics; on the four training instances the only rule that fires is the one lowering the balance weight to 0.10 under high dispersion of the machine loads, and it fires on all four, so the effective vector is the one printed with the components above, identical across the training set. The implementation also contains a generator that would derive all six weights from instance statistics, but the training entry point supplies its explicit vector and the generator is bypassed; it played no part in any experiment of this paper. These constants were fixed during early development and carried unchanged into every experiment; no claim of optimality is attached to them, and, like the transfer rule of Section 5.4, this account of their provenance was established by a post-hoc code review. For completeness, the implementation carries two dormant elements: floors on the scales that never bound on these instances, and a discontinuous terminal bonus below a 5% gap that no recorded training episode reached. The configuration campaign of Section 5.3 later raced the six weights, each free in $[0, 1]$ (explicit weights bypass the adjustment layer), and did not improve on the deployed vector within the protocol's resolution; the internal constants were not raced.

4.2. State representation

The state observed at each decision comprises two groups of features: a per-operation description of every eligible operation, scored one candidate at a time by the policy head, and a description of the shop as a whole, which enters the shared context. Operation $o = o_{jk}$ on machine $\mu = \nu_{jk}$ is described by the 16 quantities of Table 1, and the global state by the 12 aggregates of Table 2.

Both groups are dimensionless: every temporal quantity is expressed as a fraction of the computed bound $\widehat{\mathrm{LB}}$ of Section 4.1, and every uncertainty as a ratio of interval width to magnitude. Absolute times are not comparable across instances, since their magnitude reflects the time scale of the instance at hand, and a policy trained on them would be tied to the scale of its training set. The dimensionless representation renders the observation comparable across instances and sizes, which is one of the two components of size invariance; the other is the architecture of Section 4.3.

Every quantity consulted by the dispatching rules of Section 2.2 (processing times, remaining work, earliest start times, slack, machine load) appears among the per-operation features, so that no advantage over those rules can be attributed to a wider per-operation vocabulary. The representation extends that shared vocabulary in two directions: the interval bounds and relative widths of every temporal quantity, which make the uncertainty explicit, and the global aggregates, which no per-operation priority rule consults. Which of these inputs the trained network comes to rely on is measured a posteriori by the audit of Section 7.1.

Table 1: Per-operation features (all dimensionless). $d = \mathbf{p}_{jk}$, **est** = earliest start interval, **rem** = remaining work interval of the job strictly after o, excluding o itself; mid$(\cdot)$ is the interval midpoint; $L(\mu)$ the remaining worst-case load of machine μ; $\bar{L}$ its average over machines; $\mathbf{c}_J$ and $\mathbf{c}_\mu$ the completion intervals of the operation's job and machine over what is scheduled so far.

#	Feature	Definition
1–2	duration bounds	$d^L/\widehat{\mathrm{LB}}$, $d^U/\widehat{\mathrm{LB}}$
3	duration uncertainty	$(d^U - d^L)/\mathrm{mid}(d)$
4–5	earliest start bounds	$est^L/\widehat{\mathrm{LB}}$, $est^U/\widehat{\mathrm{LB}}$
6	start uncertainty	$(est^U - est^L)/\mathrm{mid}(\mathbf{est})$
7–8	remaining job work	$rem^L/\widehat{\mathrm{LB}}$, $rem^U/\widehat{\mathrm{LB}}$
9–10	job position	$(m-k-1)/m$, k/m
11	machine load	$L(\mu)/\widehat{\mathrm{LB}}$
12	potential idle gap	$\max(0,\, c_J^U - c_\mu^U)/\widehat{\mathrm{LB}}$
13	machine congestion	$L(\mu)/\bar{L}$
14	slack	$(est^U - \min_{o' \in \mathcal{E}} est_{o'}^U)/\widehat{\mathrm{LB}}$
15–16	completions	$c_\mu^U/\widehat{\mathrm{LB}}$, $c_J^U/\widehat{\mathrm{LB}}$

Table 2: Global state features (all dimensionless aggregates, notation as in Table 1; no dimension depends on n or m). Rows that list several statistics contribute one feature each, for 12 in total.

#	Feature	Definition
1	progress	scheduled operations $/\, nm$
2	partial makespan	$\max_\mu c_\mu^U/\widehat{\mathrm{LB}}$
3–4	machine completions	mean and std of $c_\mu^U/\widehat{\mathrm{LB}}$
5–7	remaining loads	mean, max and std of $L(\mu)/\widehat{\mathrm{LB}}$
8–9	remaining job work	mean and max of job remaining $/\widehat{\mathrm{LB}}$
10	eligibility	$\lvert\mathcal{E}\rvert/n$
11	pending uncertainty	$\sum$(widths)$/\sum$(midpoints) of unscheduled ops
12	total load	$\sum_\mu L(\mu)/(\widehat{\mathrm{LB}} \cdot m)$

4.3. Size-invariant architecture

The second component of size invariance is architectural. The number of candidates changes at every step and from instance to instance, while a neural network requires inputs of fixed dimension. What is fixed here is the dimension of a single candidate and not their number: the eligible set enters as a $|\mathcal{E}| \times 16$ matrix whose rows share one encoder, so the count is a row dimension that no weight matrix sees. Two elements resolve the difficulty on that basis: the *same* network reads every candidate, so any number of them can be processed, and the resulting embeddings are *pooled* into fixed-size summaries, so the scoring stage receives a representation of the whole decision. Let $x_i \in \mathbb{R}^{16}$ be the features of eligible operation i and $g_0 \in \mathbb{R}^{12}$ the global state; the construction has three stages (Figure 1).

Embed. The shared two-layer MLP ϕ (width h=128, LayerNorm, ReLU, orthogonal initialization) maps each candidate to an embedding $\phi_i = \phi(x_i)$. The whole eligible set is processed in one forward pass, as the $|\mathcal{E}| \times 16$ matrix of its rows. Sharing the weights across rows allows the stage to accept any number of candidates and makes it equivariant to their

order. Candidates do not interact at this stage: ϕ_i depends on x_i alone.

Pool into a context. The variable-size set $\{\phi_1, \ldots, \phi_{|\mathcal{E}|}\}$ must be reduced to a fixed-size vector before it can be combined with the global state. Mean and max pooling provide two complementary permutation-invariant summaries [44] which, concatenated with the global state g_0, are mixed into the context:

$$g = \mathrm{MLP}_{ctx}\Big(\Big[\tfrac{1}{|\mathcal{E}|}\textstyle\sum_i \phi_i;\ \max_i \phi_i;\ g_0\Big]\Big) \in \mathbb{R}^h. \tag{6}$$

Pooling introduces no parameters: it maps the $|\mathcal{E}| \times h$ embeddings to the $2h$ summaries the context MLP consumes. Both statistics are taken over the non-padded candidates only, since in batched training the sets are padded to the batch maximum: the mean divides by the true count and the max ignores the padded rows. This embed-then-pool pattern is the *Deep Sets* construction of Zaheer et al. [44], whose central result is that, with sum pooling and suitable choices of the two maps, any permutation-invariant function of a set admits this form. The guarantee concerns the pattern, not this network: with mean and max pooling at finite width, what is representable is an empirical matter, which the ablations of Section 7 probe. The result tables refer to the base model by this name.

Score. The *policy head* scores each pair (candidate embedding, context), and a masked softmax over the scores yields the distribution over eligible operations. The *value head* computes the state value $V(s)$ used in the advantage estimation, reading the context alone, since that expectation concerns the state rather than any one candidate:

$$\pi(i \mid s) = \mathrm{softmax}_i\, \mathrm{MLP}_\pi([\phi_i; g]), \qquad V(s) = \mathrm{MLP}_V(g). \tag{7}$$

Table 3 specifies the four trainable blocks. Each is a two-layer perceptron with LayerNorm and ReLU between its layers and a linear output; all hidden layers are initialized orthogonally with gain $\sqrt{2}$ and zero biases. The output layer of the policy head uses gain 10^{-2}, so at the start every candidate receives a nearly equal score and the untrained policy is close to uniform; the output layer of the value head uses gain 1. Pooling and the masked softmax carry no parameters, so those four blocks account for the whole network: 120,322 parameters, no dimension of which depends on n or m.

Table 3: Layer specification of the policy/value network at the hidden width h=128. Shapes are input, hidden and output dimensions.

Block	Shape	Parameters
Candidate encoder ϕ	$16 \to 128 \to 128$	18,944
Pooling (mean, max)	$\lvert\mathcal{E}\rvert \times 128 \to 256$	—
Context MLP	$268 \to 128 \to 128$	51,200
Policy head	$256 \to 128 \to 1$	33,281
Value head	$128 \to 128 \to 1$	16,897
Base network		**120,322**

Self-attention over the candidate set is the extension most commonly adopted in this literature to let one candidate condition on another, which pooling cannot express. A variant with two pre-LN transformer blocks between the embedding and pooling stages was evaluated

as a controlled ablation and yielded no improvement at either training budget, degrading the operating one by 1.4 points while carrying 3.2 times the parameters and 2.2 times the episode cost; its architecture and the full comparison are given in the supplementary material.

5. Experimental Methodology

5.1. Instances, data split and metric

Experiments use the 70 interval Taillard instances TA1–TA70 that anchor the recent IJSP literature [10, 9]: seven size classes from 15×15 to 50×20, each crisp duration replaced by an integer interval symmetric around it, with half-width drawn uniformly up to 15% of the original value [7]. The benchmark is openly available [11]. Of the 70 instances, the final model is trained on four, TA11–TA14 of the 20×15 class. Six more of the same class, TA15–TA20, form the *validation set*: never used for gradients, but every design and configuration decision was judged on their performance, so they are reported separately and excluded from claims about unseen data. The remaining 60 instances entered neither training nor validation. The genetic programming study splits the benchmark the same three ways [12], evolving its rules on TA11–TA14 and taking its design decisions on TA15–TA20, so the two families learned from the same four instances and were tuned against the same six. A second benchmark, consisting of interval versions of twelve classical instances (FT10, FT20, La21, La24, La25, La27, La29, La38, La40 and ABZ7–9) generated with the same symmetric scheme [7], is reserved for the cross-family evaluation of Section 6.3, and plays no part in training or validation either.

One convention governs every significance test reported below. The independent unit is the instance, not the pair (instance, seed): policies trained under different seeds are evaluated on the same instances, so their results are averaged over seeds before any instance is entered into a paired test. Tests are two-sided exact Wilcoxon signed-rank tests over the instances of the set in question, reported with the mean difference and its 95% confidence interval (Student t over the instance differences), and the smallest attainable p-value is 0.031 on the six validation instances and below 10^{-3} on the seventy. No correction for multiple comparisons is applied: each test is reported as an individually interpreted contrast, one per question, so the p-values are evidence about each contrast on its own and not a family-wise claim.

5.2. Computational environment

Table 4 lists the machine and the software stack. One aspect bears on the costs reported later: every number in this paper (training, evaluation, the dispatching-rule baselines and the ablations) was produced on the CPU, with the single exception of the tuning campaign of Section 5.3. At the width of Section 4.3, a forward pass is negligible next to the environment step that follows it, so the GPU yields no speedup for a single rollout; it was used only for the vectorized batched rollouts of the operating-fidelity tuning campaign (Section 5.3), where many episodes advance in lockstep and the batched forward pass amortizes. The method therefore carries no dependence on specialized hardware.

5.3. Hyperparameter configuration

All reported results use the hyperparameters of Table 5. Their values were fixed by hand during development, most of them at the defaults customary in the PPO literature. Whether

Table 4: Machine and software used for every experiment in this paper.

Component	Specification
CPU	AMD Ryzen 5 3400G, 4 cores / 8 threads, 3.7 GHz
Thread limit	2 per process (`OMP` and `MKL`)
Memory	32 GB
GPU	NVIDIA GeForce RTX 5060 Ti, 16 GB (tuning only)
Operating system	Windows 10 Pro 22H2 (64-bit)
Language	Python 3.12.6
Learning	PyTorch 2.9.1, CUDA 13.0 build
Numerics	NumPy 2.3.5, SciPy 1.17.0
Figures	Matplotlib 3.10.8; svglib 2.1.0 (diagram)
Tuning	irace 4.4.3 on R 4.6.1 (Section 5.3)

Table 5: Hyperparameters, identical across every experiment reported in this paper.

Hyperparameter	Value	Hyperparameter	Value
Optimizer		*PPO objective*	
learning rate (Adam)	$3 \cdot 10^{-4}$	clipping range	0.2
Adam ϵ	10^{-5}	GAE λ	0.95
gradient clipping	0.5	discount factor γ	1.0
Network and schedule		*PPO updates*	
hidden width h	128	epochs per update	4
weight initialization	orthogonal	minibatch size	256
policy update	every 4 ep.	entropy coefficient	0.01
greedy evaluation	every 10 ep.	advantage normalization	per update

automatic tuning would improve on them was assessed with three irace [27] campaigns of about 300 experiments each, two over the optimizer (one at reduced fidelity, one at the operating budget) and one over the six reward weights of Eq. (5): none of their winners improved on the hand-set values under a paired confirmation, whose resolution, set by the 0.46-point standard deviation across seeds (Section 6.1), is approximately one point of RE. The campaigns therefore exclude improvements larger than that magnitude, not the existence of smaller ones. Their search spaces, outcomes and by-products are provided in the supplementary material.

5.4. Training and inference

The policy is trained with PPO [34] under the hyperparameters of Table 5. Training on a set of instances proceeds in blocks, one instance at a time. Each new block starts from the weights of the *best block so far*, best being judged by the lowest best-episode makespan in raw time units; raw makespans are not comparable across instances, so this rule follows the instances' time scales as much as learning progress, and on TA11–TA14 it made the final block start from TA12's weights rather than TA13's in 26 of the 30 runs. Every block receives the same number of episodes, and the instances are presented in their benchmark order, TA11 to TA14, which the seed does not permute: it governs network initialization, action sampling

and minibatch composition, and the random streams are reseeded with it at the start of every block, not only once per run. A *rollout* is one complete construction episode played by the current policy: *sampled* when each operation is drawn from π, as during training, and *greedy* when every step takes the most probable operation. Every tenth episode (Table 5), the current policy is additionally evaluated with one greedy rollout. The *deployed artifact* of a run is the network as it stands at the end of the block with the lowest raw best-episode makespan, which on this training set was the final block, TA14, in all 30 runs; the weights of the best individual episode are tracked during training but are not the serialized artifact. Both conventions, the transfer rule and the artifact identity, were established by a post-hoc code review and are reported here as implemented: they define the system that every result in this paper measures.

The discount factor is $\gamma = 1$ rather than the 0.99 customary in deep RL [34]. The horizon here is finite and known, so future rewards can be credited in full; at $\gamma = 0.99$ the terminal makespan, the quantity being minimized, would reach the earliest decisions of a 20×15 instance attenuated to $0.99^{nm} \approx 0.05$ of itself. Updates are likewise applied over transitions pooled from four consecutive episodes rather than from one, in minibatches of 256 and with advantages normalized within each update, because the nm steps of a single episode are few and strongly correlated: pooling widens the sample the advantage estimate is computed from, and normalizing keeps the gradient scale comparable across instances of different size.

Thirty runs of this protocol were trained, differing from one another only in their random seed, and the one with the lowest mean RE on the validation instances at the deployed budget is *the policy* this paper reports; the seventy benchmark instances take no part in that choice.

Best-of-N inference.. At evaluation time the stochastic policy is exploited by drawing one greedy rollout and $N - 1$ sampled ones, and keeping the best solution under the ranking of Eq. (3), the standard sampled decoding of neural combinatorial optimization [22]. The deployed budget is N=64; results are additionally reported for a single greedy pass and, in the budget-matched comparisons of Sections 6.1 and 6.2, for N=1024. Section 7.2 measures what each budget buys.

5.5. Baselines

Constructive baselines are the dispatching rules of Section 2.2 (SPT, LPT, MOR, MWKR and EST) and the two Giffler–Thompson combinations. The results of Section 6 carry the strongest of each family, namely MOR, EST and G&T-MWKR, while SPT, LPT, MWKR and the SPT tie-break of Giffler–Thompson are dominated by a wide margin in every size class and are not reported further. The learned-symbolic baseline is the dispatching rule evolved by genetic programming for this same benchmark by Díaz Rodríguez [12], compared budget by budget in Section 6.2. The budget that is matched there is the number of schedules each method constructs, not the wall-clock time it spends: both run the same schedule builder once per sample and differ in what scores a candidate, a forward pass of the network against the evaluation of an evolved expression, which is the cheaper of the two. Matched sample counts therefore favour the policy in computational terms, and the comparison should be read as one of sample efficiency.

That asymmetry has measured magnitudes: Table 9 (Section 7.2) prices training and inference for both families in this same harness.

Table 6: RE (%) per validation instance and training budget, mean [best] over ten training runs at best-of-64 inference. The last row is the strongest constructive baseline, a single deterministic pass.

Budget	TA15	TA16	TA17	TA18	TA19	TA20	Mean	SD
100 eps	31.4 [20.4]	32.2 [16.8]	28.7 [22.3]	29.4 [16.6]	29.9 [13.7]	32.0 [23.9]	30.6	6.38
300 eps	18.9 [15.9]	16.3 [13.9]	16.4 [12.8]	19.5 [16.1]	17.6 [12.8]	15.2 [12.7]	17.3	1.50
1000 eps	**14.1** [12.1]	**12.8** [9.3]	**12.0** [10.0]	**16.0** [14.7]	**12.9** [10.1]	**13.6** [11.8]	**13.6**	**0.46**
G&T-MWKR	30.6	34.0	17.7	31.8	29.3	24.1	27.9	—

Three published IJSP metaheuristics are included as reference yardsticks rather than as direct competitors, since each searches every instance separately for minutes on dedicated hardware and reports 30-run averages: the genetic algorithm (GA) of Díaz et al. [7], the enhanced swarm artificial bee colony ESABC [9] and the filtered elitist artificial bee colony fEABC [10], the strongest of the three. Their figures are quoted from those papers, not re-run here.

6. Results

6.1. In-size performance and budget scaling

This section reports what the policy achieves on the class it is trained on, first during training and then at deployment on the validation instances.

Figure 2 shows the training dynamics at the operating budget. Within each block the episode RE falls steeply over the first ∼200 episodes and stabilizes, with no divergence in any seed; across blocks, each instance starts better than its predecessor did, the visible effect of starting each block from weights already trained on earlier instances, and the spread between seeds narrows as training advances. The levels in this figure are higher than the deployed ones because training episodes are *sampled*, with exploration active, rather than greedy.

Deployment on the validation instances is reported in Table 6, per instance and for increasing training budgets, with Figure 3 showing their means against the budget. The three budgets are paired on the same ten training runs, which differ only in their random seed, and all of them are read under the standalone evaluator. The operating budget was afterwards extended to the thirty runs of the campaign, whose mean is 13.92% with a standard deviation of 0.60. Three observations follow. (i) The policy scales monotonically with budget (30.6% → 17.3% → 13.6% mean RE at 100/300/1000 episodes per instance), with the return per tripling of the budget falling from 13.3 points to 3.7. (ii) The spread between training runs contracts as the budget grows, the standard deviation across the ten seed means falling from 6.38 points at 100 episodes to 1.50 at 300 and 0.46 at 1000, a factor of fourteen, so what the extra budget yields is not only a better mean but a more reproducible one; this is the deployed counterpart of the narrowing seed band of Figure 2. (iii) The gap to the strongest constructive baseline is a factor ≈2.0, and it is opened by the budget: at 100 episodes the policy is still behind that baseline. Figure 3 places the curve between two dotted references measured on the same six instances: G&T-MWKR above, at 27.9%, and fEABC below, at 9.6%, the latter a reference from per-instance search.

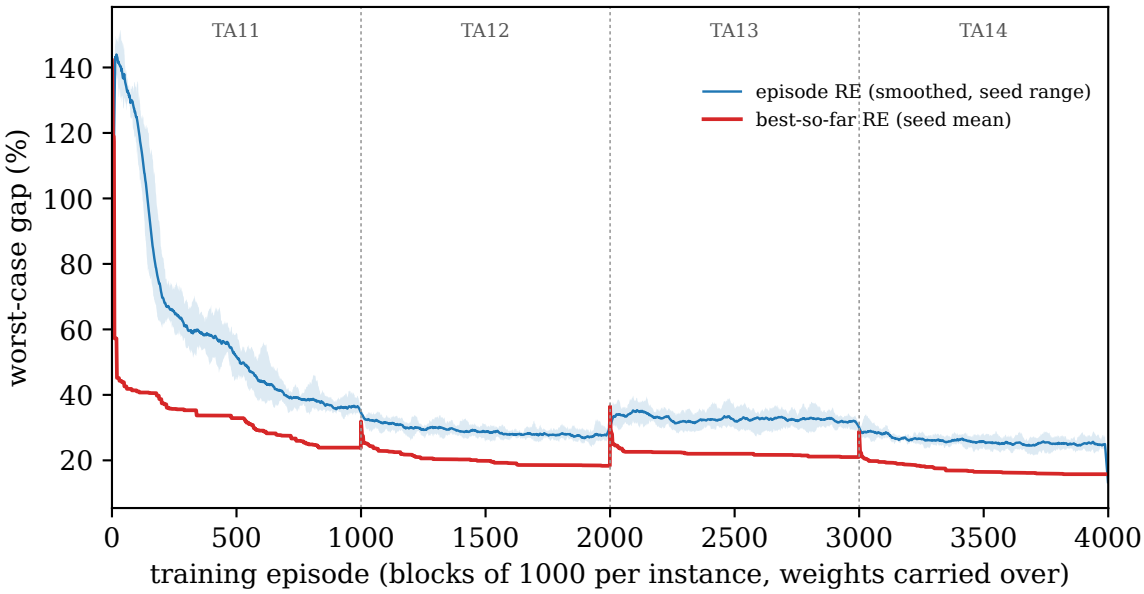

Figure 2: Training dynamics of the deployed arm: worst-case gap to the instance bound of the sampled training episodes (smoothed over 25 episodes; the band spans the seeds) and of the best schedule found so far, over the four training blocks TA11–TA14. The training logs record the worst-case makespan, the quantity the reward optimizes, so the axis is not the midpoint RE of Eq. (4).

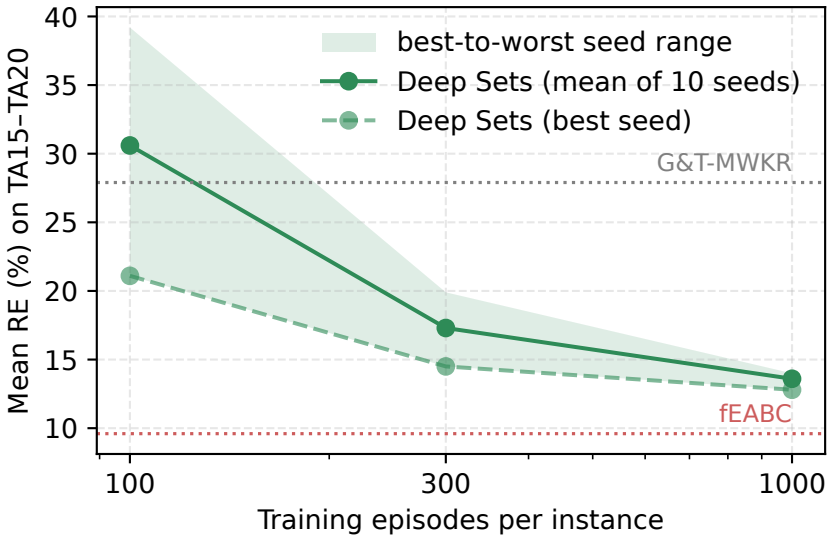

Figure 3: Mean RE on the six validation instances TA15–TA20 (20×15) against the *training* budget, the episodes spent once per instance before deployment, on a logarithmic axis and over ten training runs. The band spans the best and worst of them; the dotted lines are G&T-MWKR and fEABC on the same six instances.

6.2. Cross-size generalization and position among constructive methods

The size-invariant design allows direct evaluation of the trained network on any class. Six of the seven are *zero-shot*: the weights are applied to sizes they never saw, with no additional

Table 7: Mean RE (%) per size class over the 70 interval Taillard instances. The 20×15 column (†) holds the training and validation instances. GP and policy rows carry the artifact each study selects among its thirty.

Method	15×15	20×15†	20×20	30×15	30×20	50×15	50×20	All
EST	32.3	49.2	41.5	42.4	56.6	33.2	41.0	42.3
MOR	41.4	46.7	47.3	44.1	58.8	34.1	45.9	45.5
G&T-MWKR	26.7	29.0	30.9	33.6	36.8	23.9	25.5	29.5
GP rule, 1 pass	15.7	16.6	18.1	21.1	24.1	13.8	14.6	17.7
Policy, 1 pass	15.0	17.4	17.7	21.4	24.7	16.1	17.1	18.5
GP rule, 64	12.6	15.9	16.8	17.4	23.4	11.7	13.4	15.9
Policy, 64	10.7	12.8	14.1	17.6	22.3	12.8	14.8	15.0
GP rule, 1024	10.2	14.0	15.1	15.5	21.0	10.6	12.1	14.1
Policy, 1024	9.3	11.2	12.2	15.6	19.9	10.9	13.7	13.2
fEABC (30 runs)	5.9	8.7	10.3	9.8	16.4	5.7	9.0	9.4

training or fine-tuning of any kind. The seventh, marked in the table, is 20×15, which holds the four training and the six validation instances. Table 7 reports RE over all seventy instances by size class, ten to a class, with its per-instance values in the supplementary material. Each learned family trains thirty artifacts and appears as the one its study selects: the rule Díaz Rodríguez [12] features and the policy of Section 5.4. Both were fitted on the 20×15 instances, so the other six classes lie outside the training distribution of either and the transfer the table asks for is the same on both sides. The GP rule is deterministic at one pass and randomized by ϵ-greedy dispatching when sampled; fEABC closes the table as a 30-run average from a different computational class.

At every budget the policy improves on G&T-MWKR, the strongest of the hand-crafted baselines considered here, on *every* one of the seventy, with class means at 64 samples from 10.7% on 15×15 to 22.3% on the hardest transfer, 30×20, and no failure mode up to 1000-operation episodes. That is a property of the selected run rather than of the arm: across the thirty runs at one pass, 64 of the 2100 (instance, run) pairs fail to improve on the same rule, three per cent of them.

Against the GP rule the comparison is not one-sided: its outcome depends on the inference budget both methods receive (Figure 4). With one pass each the rule is better, 17.71% against the policy's 18.49%, which is ahead on only 26 of the 70; with 64 samples each the ordering reverses, 15.02% against 15.88% and ahead on 42 of the 70, and 1024 samples widen nothing, 13.25% against 14.07% and again 42. With the instance as the experimental unit (Section 5.1), every one of these differences is significant ($p = 0.028$, 0.020 and 0.010): the two learners are statistically distinguishable at every budget, in opposite directions, and the advantage the policy gains by sampling is of the same size as the one it concedes at a single pass.

The deficit does not depend on which run each side put forward. The means over the thirty, which no choice touches, stand 18.99% against 19.82% and separate with $p = 0.006$, so the ordering at one pass is a property of the two families and not of the two artifacts that represent them. Selection itself is asymmetric between the studies: the policy is chosen

on the validation instances, while the GP study features the rule with the best mean over these seventy. The asymmetry is immaterial here, because selecting the rule on the validation instances returns the same artifact (17.45% there, against 18.13% for the runner-up), and the thirty-run means involve no selection at all; but the selected-pair contrasts should be read with it in mind.

The per-class columns qualify the aggregate. The one-pass deficit is not spread evenly: the policy is *better* on 15×15 and 20×20, and the whole of the aggregate gap comes from 50×15 and 50×20, where it loses by 2.3 and 2.5 points; the means over the thirty artifacts follow the same pattern. Both families are size-agnostic in form, the rule as an expression over per-operation attributes and the network by construction, so what separates them on the largest classes is not applicability but capacity. The network fitted its decision function to the states a 20×15 episode visits, and those move with size: a 50×20 episode runs more than three times longer and offers up to fifty candidates at a step rather than twenty. A short expression has little room to encode a dependence on that distribution, and correspondingly little to lose when it shifts. Sampling recovers most of the gap, but the two largest classes remain where the evolved rule is hardest to beat.

The network's advantage therefore lies in its distribution, whose samples improve faster with budget than those of the randomized rule. The best-of-N ablation of Section 7.2 reaches the same conclusion from the training side. The policy's standalone RE (13.9% in-size, 9–20% class means at the largest budget) places it between the dispatching rules (42.3% for EST, the strongest) and the per-instance metaheuristics (9.4% for fEABC). This is the ordering that Zhang et al. [46] established for the deterministic JSP, in which a learned constructive policy improves on the hand-crafted rules but does not reach the methods that search each instance anew, trading some solution quality for a cost incurred once at training rather than at every instance; it had not been established for the interval variant.

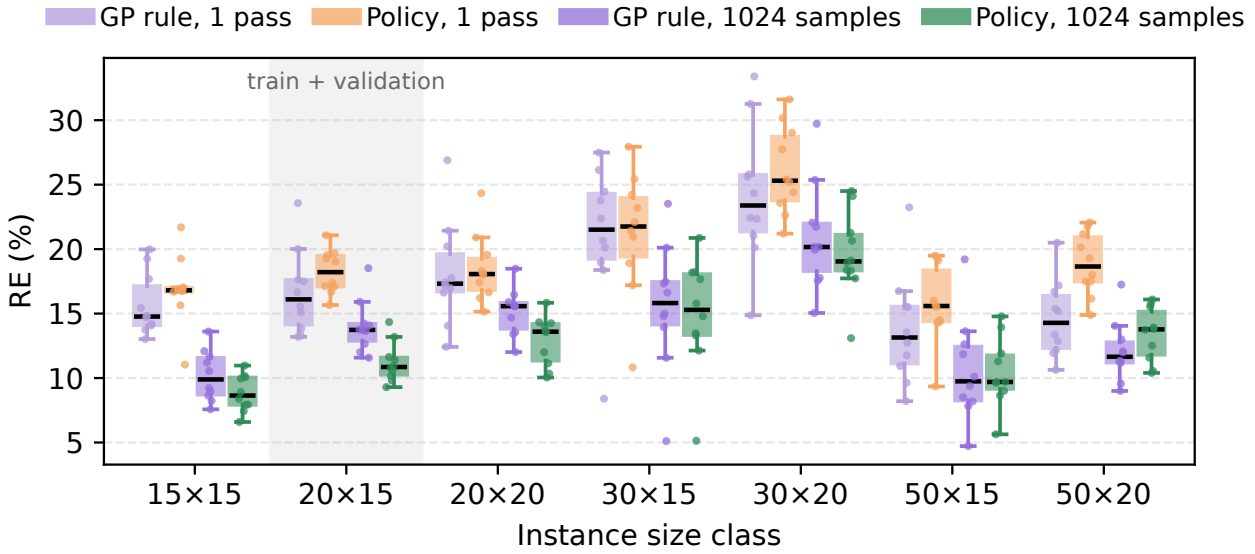

Figure 4: RE by size class over the 70 interval Taillard instances, the two learners paired at one pass and at 1024 samples. Boxes span the quartiles with whiskers at 1.5 IQR; every instance is overlaid as a point. The shaded 20×15 class holds the training and validation instances.

6.3. Cross-family generalization: the classical instances

The IJSP literature also reports on interval versions of twelve classical instances (FT10 and FT20, seven of the La family, and ABZ7–9), generated from their crisp counterparts with the same symmetric scheme as the Taillard set [7]. None of these families was seen during training or validation, so they double as a cross-family generalization test. What changes is the family of underlying scheduling instances, not the way uncertainty is generated, which is the same construction in both benchmarks; the test therefore speaks to transfer across shop structures and leaves transfer across uncertainty regimes open. Following the literature, RE is measured here against the best-known expected-makespan values [10].

Table 8: RE (%) on the 12 classical interval instances, by inference budget. Entries are means over thirty runs, with the per-instance best over those runs in brackets.

	rule 1 pass	learned one pass		learned 64 samples		learned 1024 samples		search 30 runs
Inst.	G&T	GP	**Pol.**	GP	**Pol.**	GP	**Pol.**	ESABC
		mn [bst]	mn [bst]	mn [bst]	mn [bst]	mn [bst]	mn [bst]	mn
FT10	32.2	17.0 [7.2]	12.8 [6.6]	11.8 [5.2]	7.9 [4.3]	8.5 [4.2]	6.1 [4.3]	3.0
FT20	40.0	18.5 [9.4]	12.2 [3.4]	13.3 [5.8]	8.1 [3.4]	10.3 [5.4]	6.2 [2.4]	1.8
La21	24.1	15.9 [10.3]	18.1 [12.4]	12.6 [10.0]	12.5 [8.6]	10.7 [6.8]	9.5 [6.7]	4.0
La24	26.3	16.1 [10.1]	16.6 [9.2]	11.5 [8.6]	11.2 [8.8]	9.2 [6.2]	9.3 [5.9]	5.0
La25	16.9	17.6 [9.6]	18.4 [8.3]	12.3 [8.7]	10.3 [7.8]	9.3 [5.1]	8.2 [5.8]	2.7
La27	34.8	18.3 [12.3]	14.9 [8.9]	12.5 [10.7]	11.4 [8.4]	11.3 [8.2]	9.3 [7.7]	4.1
La29	24.5	19.9 [12.9]	22.4 [13.2]	15.0 [11.4]	15.9 [12.3]	12.6 [9.7]	13.5 [7.8]	7.0
La38	27.0	17.4 [9.2]	15.8 [10.8]	14.3 [9.2]	11.1 [8.0]	12.0 [7.1]	9.1 [7.0]	5.8
La40	27.9	16.7 [11.7]	13.4 [9.3]	12.3 [8.8]	8.5 [6.0]	10.0 [7.1]	7.2 [4.5]	4.1
ABZ7	28.7	17.6 [13.3]	18.2 [14.1]	14.3 [11.7]	13.5 [10.4]	12.2 [8.9]	11.6 [8.5]	6.7
ABZ8	36.9	24.6 [18.1]	21.4 [16.8]	20.9 [17.0]	17.5 [15.0]	18.6 [15.0]	15.3 [12.6]	10.9
ABZ9	41.8	29.0 [19.5]	24.7 [19.7]	24.0 [17.9]	19.5 [16.3]	21.5 [17.4]	17.3 [13.2]	11.2
Mean	30.1	19.0 [12.0]	17.4 [11.0]	14.6 [10.4]	12.3 [9.1]	12.2 [8.4]	10.2 [7.2]	5.5

The per-instance metaheuristics are quoted from Table 2 of Díaz et al. [10] and belong to a different computational class. The policy improves on EST and G&T-MWKR on all twelve instances and lands at 12.3% mean RE at its deployed budget, between the hand-crafted constructive baselines (30.1% for G&T-MWKR) and the per-instance searchers, represented in the table by ESABC at 5.5%. Given 1024 samples the mean over its thirty runs reaches 10.2%, within half a point of the 30-run average of the weakest of those searchers, the GA of Díaz et al. [7] at 9.8%, a method that searches each of these instances from scratch while the policy has never seen them; ESABC stays clear of both learners.

Against the evolved rules the comparison is again made budget by budget, and Figure 5 is arranged that way. The policy holds the lower figure in all six cells, and the two readings agree at every budget. Given one pass each it leads by 1.64 points on the means and 0.92 on the brackets, ahead on 7 of the 12 instances, a margin this sample cannot separate ($p = 0.09$ and $p = 0.27$). Given 64 samples each, with the rules randomized by the ϵ-greedy dispatching their own study provides, it leads by 2.28 and 1.31 points, ahead on 11 and 10 of the 12 ($p = 0.003$ and $p = 0.005$); at 1024 samples, by 1.96 and 1.21, ahead on 10 of the 12 in both readings ($p = 0.003$ and $p = 0.012$).

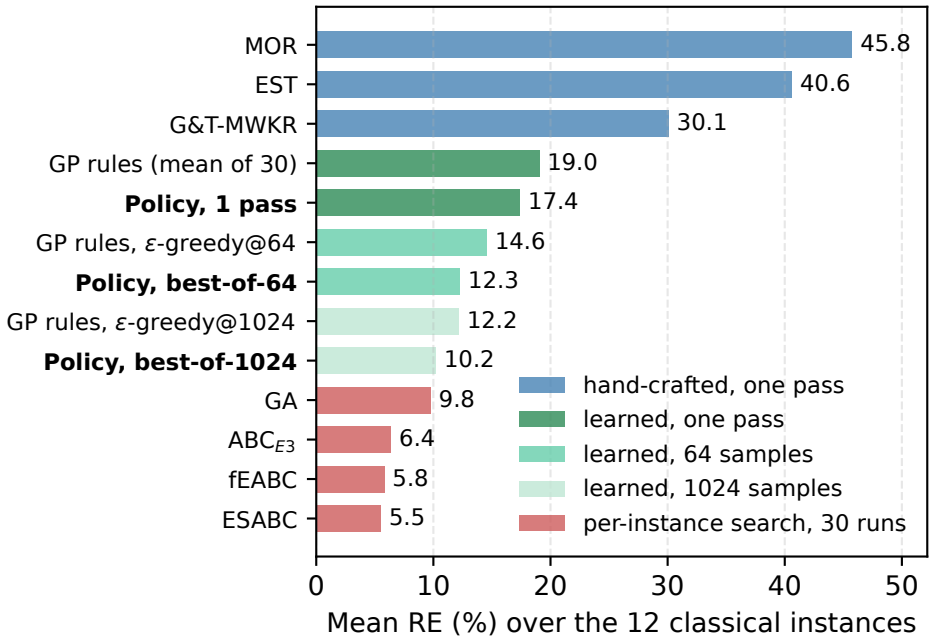

Figure 5: Mean RE over the 12 classical interval instances, grouped by the inference budget each method is given. Both learned entries are means over the thirty independent runs of their family.

The ordering therefore differs from the one the Taillard set gives. There the rule was ahead at a single pass; here it is behind at every budget, though the one-pass difference does not reach significance. Size is one difference between the two tests. These twelve instances range from 10×10 to 20×15, so none of them exceeds the class both learners were trained on, whereas the Taillard set reaches 50×20. The classes on which the rule overtook the policy there are therefore absent here.

7. Analysis

Three questions remain about the policy itself: which of its inputs it consults, which of its design choices earn their cost, and how far the makespans it predicts survive execution.

7.1. What the policy attends to

The policy is not interpretable the way a priority expression is, but its inputs can be audited. Figure 6 reports a permutation test [5]: at every decision of a greedy rollout, one feature column is shuffled across the eligible operations, and the resulting mean RE over the validation set and the first three training seeds is compared with the intact rollouts (18.2%).

Each column is permuted five times with independent draws, and the figure reports the mean degradation and its standard deviation over the five; a single draw moves the magnitudes by several points and reorders the lower half, though not the leading three. Three features carry the policy: the relative slack (whose permutation adds 56.5 ± 2.4 points of RE), the number of remaining operations in the job ($+33.6 \pm 1.4$), and the job's worst-case remaining work ($+9.6 \pm 1.0$). These are the attribute families behind MOR and MWKR,

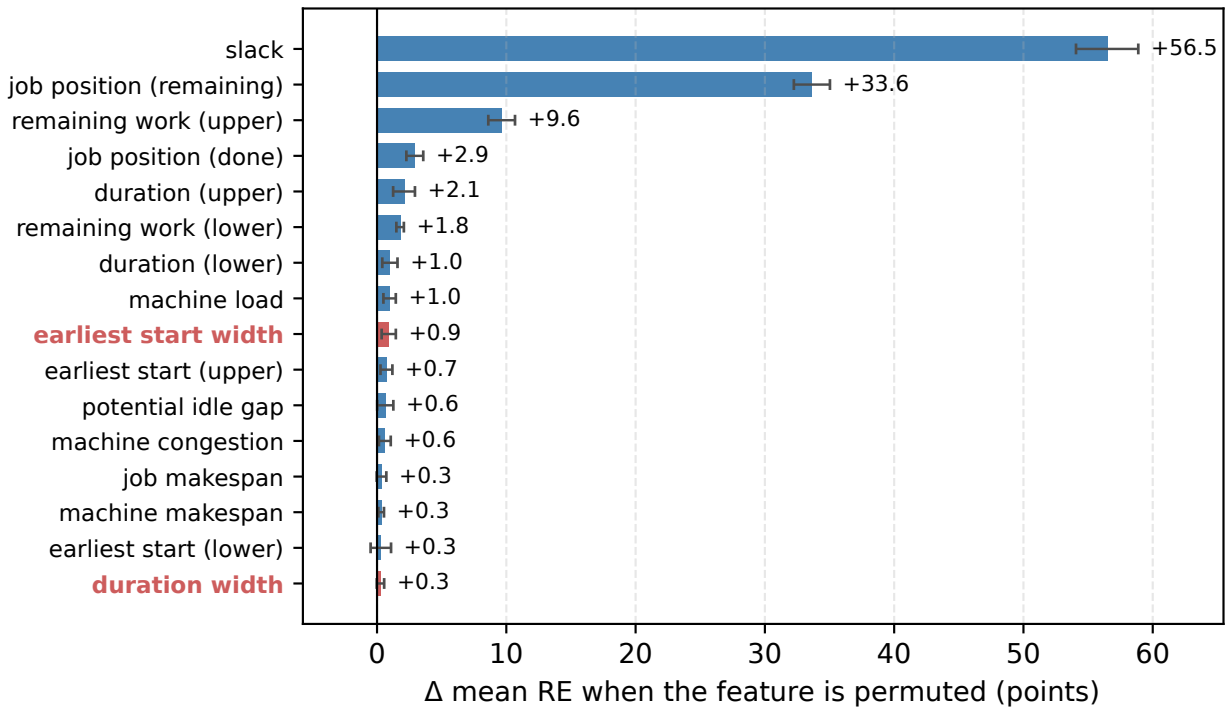

Figure 6: Permutation importance of the 16 operation features, as the change in mean validation-set RE. Bars are the mean of five independent permutation draws and whiskers their standard deviation; the two interval-width features are highlighted.

and the very attributes the evolved expressions of Díaz Rodríguez [12] select. Below a second tier of three features costing between 1.8 and 2.9 points, the remaining ten fall into a floor between +0.3 and +1.0 points that no measurement of this precision separates. The floor is not exactly zero because permuting any column perturbs an input distribution the network was trained against, whether or not that column informs the decision.

The two interval-width features sit inside that floor, one of them last of the sixteen ($+0.3 \pm 0.3$ for the duration width, $+0.9 \pm 0.5$ for the start width), and their position is consistent with the selection objective. The primary key of the criterion of Eq. (3), the worst-case makespan, on which the terminal reward trains, is a function of the upper endpoints alone (Section 7.3), so a width can influence the *selection* only through the secondary key, that is, through exact ties of the upper endpoint, and Section 7.3 measures how rarely such ties decide, in seven of its three hundred rollout pools. The training signal is less pure, one shaping term reading a lower endpoint (Section 4.1), which makes the floor position of the width features a measured outcome rather than a foregone one. The same asymmetry appears within the features themselves: three quantities enter as a pair of interval endpoints, and in the two pairs that rise above the floor the upper endpoint costs several times its lower counterpart (9.6 against 1.8 for the job's remaining work, 2.1 against 1.0 for the duration), as a criterion written in upper endpoints alone should induce.

The audit is therefore conditional on what the policy was trained to optimize, namely this objective and the reward weights of Section 4.1, and a different one need not produce the same ranking. The corresponding test is an objective that does contain the width, which

Section 7.3 carries out: retraining under f_λ leaves the weights indistinguishable from the default arm's under a common selection criterion, which is evidence that the ranking would survive that change, though the audit itself was not repeated on those arms.

7.2. Ablation: best-of-N inference

Section 6.2 samples the budget axis at three points, so it locates the reversal between the two learners no more precisely than between one pass and 64. This section reconstructs the whole curve and finds the crossing near its left end: the policy improves on its own greedy pass from B=2 and overtakes the evolved rule at B=6, so sampling becomes profitable at budgets far below the 64 used throughout, and the rule's advantage at one pass occupies a narrow initial segment of the budget axis. The reconstruction retains both endpoints of all 342 rollouts of each of the ten training runs on each of the 70 instances (239,400 schedules), so that best-of-B under the deployed protocol, the greedy pass plus $B-1$ samples retained by the criterion of Eq. (3), can be read off for arbitrary B without further evaluation. Each run spends its own budget, as a deployed policy does, and the curve is the mean over the ten; Figure 7 plots it with a band spanning the best and worst run, the dispersion a practitioner training once would face. EST, at 42.3%, lies above the frame. A single *sampled* rollout is worse than the greedy one, 21.3% against 19.5%: the argmax is a high-quality sequence within a broadly dispersed distribution. The selection key is immaterial to the level of the curve: retaining by the midpoint instead of the lexicographic key lowers the reported RE by at most 0.17 points anywhere on the axis, so the curve measures the budget and not the tiebreaking convention. Past the crossing the returns are near-logarithmic, 15.3% at 64 and 14.2% at 341, without an inflection: no budget within the tested range saturates. The level agrees with the independent measurements of Table 7, where the selected run reaches 15.0% at the same budget, slightly ahead of this ten-run mean as the best of thirty should be.

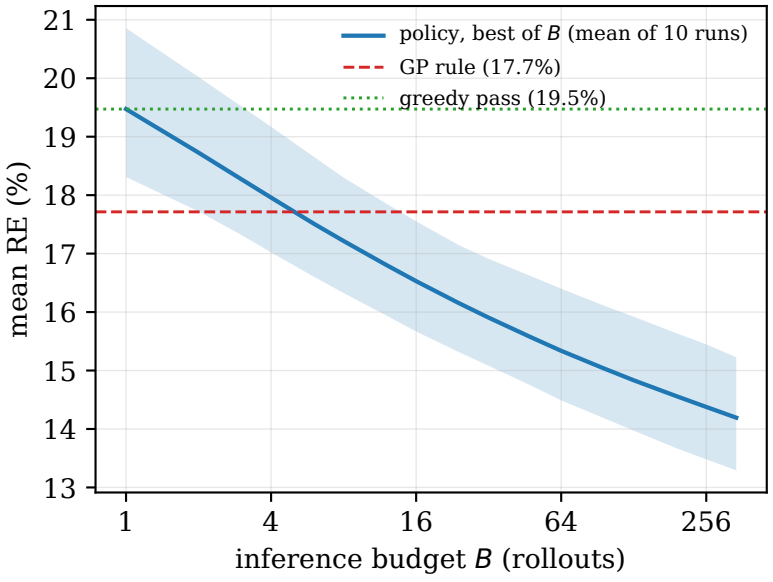

Figure 7: RE against the *inference* budget B on a base-2 logarithmic axis: the mean over the 70 instances of the schedule a run retains under the deployed protocol, its greedy pass plus $B-1$ samples kept by the criterion of Eq. (3), averaged over the ten training runs. The band spans the best and worst run.

The same reconstruction quantifies the marginal return of further sampling and its computational cost. The marginal return decays slowly but does not vanish, from 0.57 points between 32 and 64 samples to 0.46 between 128 and 256, a rate of about half a point per doubling that is itself contracting by roughly a tenth per octave; wall-clock cost, by contrast, scales linearly in B. On the machine of Table 4 a sampled rollout requires 1.03 s on a 20×15 instance and 6.4 s on a 50×20 one, timed end to end over the episode, so that the figure covers the forward passes, the environment transitions and the interval arithmetic of the schedule builder, and excludes only the one-off loading of the model. Accordingly the 1024 samples of the largest budget reported amount to 18 minutes and 1.8 hours per instance, and closing the residual gap to fEABC's 9.4% would take some ten further doublings at the measured rate, which the same arithmetic prices in months. Sampling is therefore an inefficient route to that gap, which fEABC's own study reports attaining in minutes per instance on the hardware of that study. The extrapolation is the optimistic reading twice over: budgets above the measured range are outside it, and within it the return per doubling is already shrinking.

Table 9: Wall-clock cost of the two families on the machine of Table 4.

	GP rule	Policy
Training one artifact	26 min	92 min [60–153]
Thirty artifacts	4.4 h	16 h
One deterministic pass, 20×15	0.10 s	0.86 s
50×20	1.00 s	3.98 s
Per sample at best-of-64, 20×15	0.10 s	1.03 s
50×20	0.93 s	6.4 s

Table 9 closes the budget analysis by pricing both families in this harness, on the same machine, the same representative instance per class and, for the training rows, the same three concurrent processes. Evolving one rule takes 26 minutes against a median of 92 for training one policy, 4.4 against 16 hours for the thirty artifacts of each family, both costs incurred once. Per schedule, with the best-of-64 batch timed whole and divided by 64, the rule is about seven to eleven times cheaper depending on the size, and ϵ-greedy sampling adds nothing measurable to its deterministic pass. Both families run the same schedule builder, so the factor is the forward pass against the evaluation of a compact expression. The sample-matched protocol of Section 6.2 charges for neither difference, which is the premium behind the sample-efficiency reading fixed in Section 5.5.

7.3. Where each paradigm handles the uncertainty

Componentwise addition and join act on each endpoint separately, so the recursion of Eq. (1) splits in two. The upper half,

$$c_o^U = \max\left(c_{JP(o)}^U,\, c_{MP(o)}^U\right) + p_o^U, \tag{8}$$

makes $C_{\max}^U$ a deterministic function of the processing order and the upper endpoints alone, the lower half determining $C_{\max}^L$ without ever entering it. $C_{\max}^U$ is the primary key of the lexicographic criterion of Eq. (3), whose secondary key, the lower endpoint, is consulted only

on exact ties of the upper one; everything below is therefore conditional on such ties deciding rarely, and how rarely is measured rather than assumed. Ties of the upper endpoint are common, every one of the 300 (arm, instance, seed) rollout pools of this section containing some, but only a tie at the *minimum* hands the selection to the secondary key, and that happens in 7 of the 300 pools; where it does, the key chooses among schedules with the same worst case, so it sets the width of the reported schedule, never the objective. The training signal is a different matter. The terminal reward of Section 4.1 trains on that primary key, but one of its shaping components, the idle penalty, charges the worst case of an idle interval, which reads the machine's lower completion endpoint (Section 4.1): a channel through which interval information beyond the upper endpoints reaches the weights does exist. Whether the width features inform the trained policy is therefore an empirical question rather than a corollary of the objective, and the audit and ablations below answer it.

The permutation audit of Section 7.1 found both width features at the floor of its ranking, consistent with this entailment. Three training-time ablations ask the causal questions the audit cannot: whether the policy would be any worse had it never seen the uncertainty at all; whether the upper bounds shown during training must be the true ones; and whether the policy would come to consult the widths under an objective that pays for them. All three share one design: ten retrained seeds per arm, each paired with the same ten seeds of the main arm, the standalone evaluator, and the instance as the experimental unit. What varies is only where the sampled evaluation, which costs 64 rollouts per instance, can be afforded: at that budget the first two ablations are read on the six validation instances and on thirty unseen ones, five per size class fixed by benchmark index before any result was seen, while the cheap single-pass reading covers all sixty. The third ablation probes the selection stage itself, so it is read on deposits that retain all 64 rollouts per (arm, instance, seed); under the paper's selection criterion the main arm's deposits stand at 13.96%, the baseline of every figure in that analysis.

Widths as inputs.. The first ablation removes the information: the encoder collapses every interval to its worst case before the features are computed, so the width features are identically zero and the bounds coincide, while the architecture, the hyperparameters and the schedule builder remain untouched and the environment still executes interval semantics. No set separates the two arms: 13.98% against 13.57% on the validation instances (0.42 points, $p = 0.094$), an indistinguishable 15.53% against 15.55% on the thirty unseen ones (worse on 15 of the 30, $p = 0.90$), and nominally ahead by 0.24 points at one pass over the sixty ($p = 0.10$). The measurement is not redundant with Eq. (8), which constrains the objective but not the value function or the exploration the widths might have shaped; they shape neither, and the thirty unseen instances bound what room remains between -0.34 and $+0.29$ points with 95% confidence.

Intervals as training data.. The second ablation removes the uncertainty from training altogether. The benchmark perturbs each crisp duration symmetrically, so the original crisp instances *are* the midpoint scenario of their interval counterparts: the policy is trained on them and evaluated, unchanged, on the interval instances. This is the one case Eq. (8) leaves open, since midpoint training replaces the upper endpoints themselves. Here the sets do not agree, and the disagreement is informative. At the deployed budget the gap is present on both: 0.57 points on the validation instances, from 13.57% to 14.14% ($p = 0.063$), and 0.43

points on the thirty unseen ones, from 15.55% to 15.99% (95% CI 0.03 to 0.84), worse on 21 of the 30 ($p = 0.045$). At one constructive pass over sixty unseen instances it vanishes (-0.04 points, $p = 0.65$). Training on the true upper bounds is therefore better than training on their midpoints when the policy samples, and not measurably so when it constructs once, which places the contribution of the interval data at the same stage the objective ablation below identifies. The two halves have different reach. On the selection side, Eq. (8) holds for any method that ranks semiactive schedules by Eq. (3): widths enter selection only through the two per cent of tied minima, whatever the model class. On the training side no such entailment is available, since this reward carries a lower-endpoint term; the inertness of widths as *inputs* and the value of intervals as *training distribution* are both empirical facts about this reward and this benchmark, measured above.

A width-penalizing objective.. The third ablation changes the objective so that the width enters it. The policy is retrained, unchanged in every other respect, under

$$f_\lambda \;=\; C^U_{\max} + \lambda \left(C^U_{\max} - C^L_{\max} \right), \qquad \lambda = 1, \tag{9}$$

which charges the schedule for the width of the interval it predicts and is the analogue of the robust objective of the GP study [12]; since optimizing one quantity and selecting by another would be incoherent, the best-of-N inference of this arm ranks its samples by f_λ as well. One caveat from the post-hoc code review of Section 5.4 is specific to these arms: the training-side tracking key that judges the best block reads the lower endpoint of the lexicographically largest job completion rather than the componentwise maximum of the lower endpoints, so the f_λ it tracks can differ from Eq. (9) (the upper endpoint, and with it every $\lambda = 0$ arm, is unaffected); all reported evaluations and every number in this section use the componentwise makespan. Deployed as that package, the objective delivers what it promises: over the ten seeds the predicted interval narrows from 13.05% to 12.18% of the expected makespan (narrower on all six instances, $p = 0.031$) while the expected makespan rises by 1.15 points, to 15.11% (higher on all six, $p = 0.031$). Unlike the two ablations above, both effects are significant: a width-rewarding objective is available, at a price of about one point of RE at this λ.

Figure 8 locates where that trade is produced. Training and selection changed together in the deployed package, so the two are separated on the common deposits, with three further arms retrained from scratch at $\lambda = 0.5$, 2 and 4, ten seeds each. When deployed, with each arm's samples ranked by its own f_λ, the five arms trace a monotone frontier, from 13.05% relative width at the default end to 11.08% at $\lambda = 4$, with the expected makespan climbing by 3.40 points and every step of the narrowing significant ($p = 0.031$, all six instances at every λ). Under a common selection criterion the frontier vanishes: no retrained arm proposes narrower rollouts than the default one, the four widths falling within a tenth of a point of the default's 13.05% (at $\lambda = 1$, 12.98% against 13.05%, $p = 1.00$; smallest $p = 0.44$), and the RE differences are equally null (between -0.07 and $+0.47$ points, smallest $p = 0.16$). Conversely, re-ranking the *default* policy's own deposit by each f_λ in turn retraces the frontier almost point for point, ending at 11.04% width and 17.16% RE where the arm trained on f_4 lands at 11.08% and 17.36%. What narrows the interval is which sample is kept, not which weights produced it: the retrained weights propose samples as wide as the default's, and the default's own samples suffice to trace the frontier. The predictability–performance trade is real, tunable and monotone, and it arises entirely at selection time.

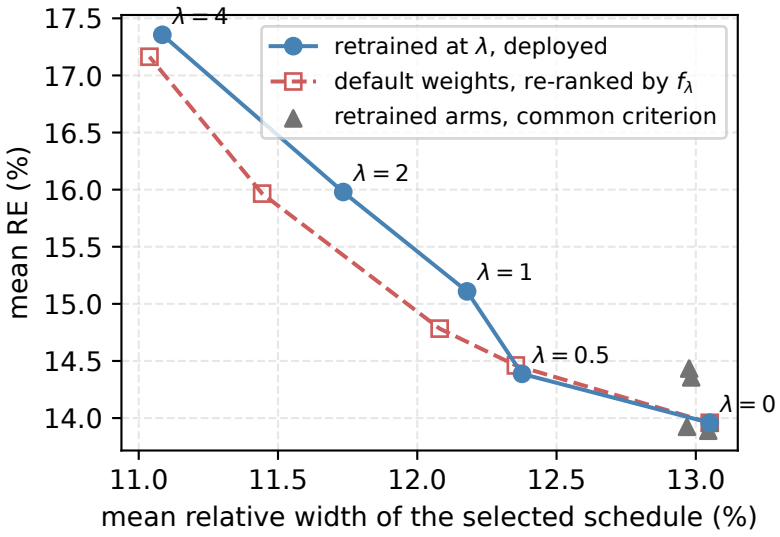

Figure 8: Mean relative width of the selected schedule against its mean RE, over the 60 (instance, seed) pools of each arm's common rollout deposit, under three selection regimes.

Contrast with the evolved rule.. The GP study runs the same objective at the same λ and reaches the opposite finding. There, penalizing width narrows the schedules only when the rule has width terminals to act through: sweeping λ from 0 to 4 moves the relative width by 1.64 points for rules that have them and by 0.13 for rules that do not, from which that study concludes that the objective works *through* the representation [12]. The network of this paper has both the terminals' equivalent and the objective, and still its weights do not move at any λ; what moves is the ranking among its samples. The same reward therefore reaches the decision function of an evolved rule and not that of a trained policy, at least at this training scale. The rate of the trade is comparable on the two sides, about 2.5 points of RE per point of width across that study's sweep and about 1.7 across ours, so what separates the paradigms is the stage at which the cost is incurred: in GP the decision function, the evolved rule becoming width-averse in every schedule it constructs; in DRL the selection, the cost arising whenever the ranking passes over a rollout with a better expected makespan in favor of a narrower one. At deployment the difference is operational: retuning a width-averse rule means evolving a new one, whereas the policy's trade-off is a parameter of the ranking, adjustable at inference time over the same samples. Two explanations remain open: that no training signal reaches these features, or that the semiactive constructive setting fixes the width independently of the policy's decisions.

7.4. Executional robustness

The evaluations so far concern the quality of a predicted schedule rather than its reliability. A schedule is committed as a processing order before the durations are known and is eventually executed under one realization of them, and its prediction is useful insofar as the realized makespan stays close to the expected value reported in advance. This is quantified

with the field's $\bar{\varepsilon}$ measure [26, 7]: for a fixed processing order with predicted makespan $\mathbf{C}_{\max}$,

$$\bar{\varepsilon} = \frac{1}{K} \sum_{k=1}^{K} \frac{\left| C_{\max}^{\mathrm{ex},k} - E[\mathbf{C}_{\max}] \right|}{E[\mathbf{C}_{\max}]}, \tag{10}$$

where $C_{\max}^{\mathrm{ex},k}$ is the crisp makespan of executing the order under the k-th realization, durations drawn independently and uniformly within their intervals. $\bar{\varepsilon}$ is thus the mean relative distance between the executed makespan and the reported value, a measure of the calibration of the prediction, in which lower is more faithful. The measurement uses K=1000 common random numbers per instance (identical realizations for every method, drawn from a generator seeded by the instance identifier so that the draws reproduce across runs and machines), so every comparison below is paired realization by realization. The measurement covers the full seventy-instance benchmark.

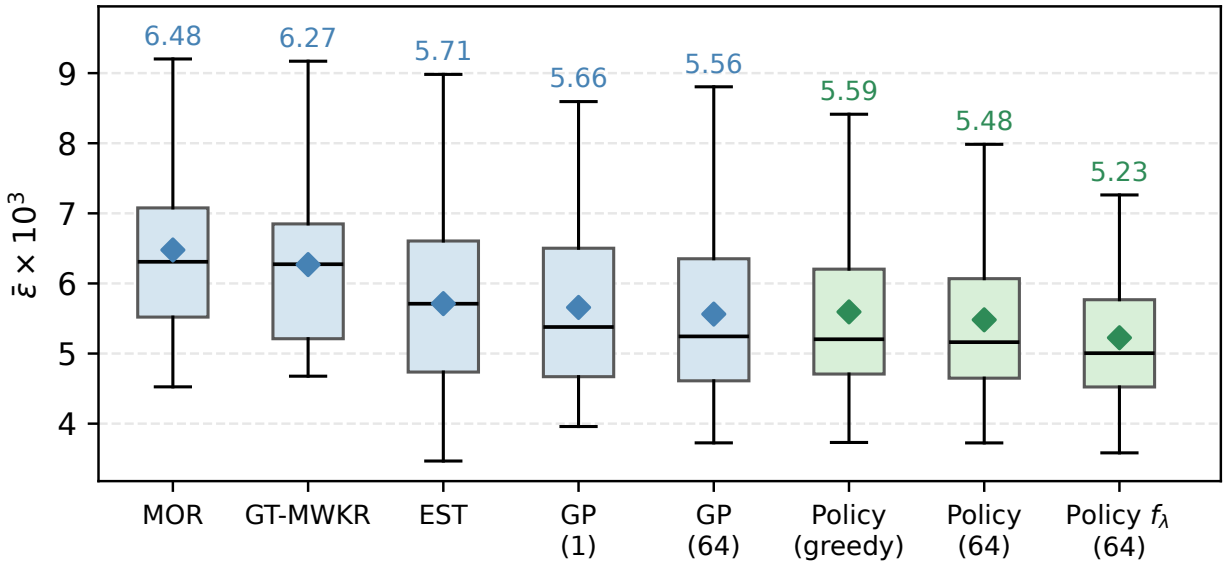

Figure 9: Executional robustness by method: $\bar{\varepsilon} \times 10^3$ over the seventy benchmark instances; lower is a more faithful prediction. Boxes span the quartiles with whiskers at 1.5 IQR, and diamonds mark the means quoted in the text; the number in parentheses is the inference budget in constructed schedules.

The constructive methods separate into two groups (Figure 9). MOR and G&T-MWKR deviate by $\bar{\varepsilon} \times 10^3 = 6.48$ and 6.27; every remaining method lies between 5.2 and 5.7, and the distance is systematic: the policy at best-of-64 is more faithful than MOR on 63 of the seventy instances and than G&T-MWKR on 65 (paired Wilcoxon signed-rank, $p < 10^{-10}$ in both cases). Within the faithful group the policy's best-of-64 (5.48) is more faithful than EST (5.71; 45/70, $p = 0.010$) and level with the evolved rule of Díaz Rodríguez [12] at both of its budgets (5.66 at one pass, $p = 0.069$; 5.56 at 64 samples, $p = 0.39$). Sampling degrades calibration for neither learner: the policy's best-of-64 deviates as its greedy pass does (5.48 against 5.59, $p = 0.29$), and the rule under the ϵ-greedy dispatching of its study as its deterministic pass does (5.56 against 5.66, $p = 0.54$).

The composition of the leading group explains the ranking. It holds a hand-crafted rule and both learners at their two inference budgets, and the presence of EST, the weakest

method of this paper in solution quality at 42.3% mean RE (Table 7) yet well clear of MOR and G&T-MWKR in faithfulness, shows that faithfulness and quality are separate properties: a method can predict reliably while scheduling poorly, and optimizing the makespan secures no reliability in return. The mechanism is the interval itself. Every realization's makespan lies within the predicted interval (Section 3), so the width of that interval bounds the deviation any execution can produce: the narrower the prediction, the smaller the largest deviation it admits. None of the methods in the leading group optimizes the width of the interval it predicts, so their faithfulness is bounded by whatever widths their makespan-directed construction happens to produce. Faithfulness must therefore be pursued deliberately rather than inherited from a makespan objective; the GP study reaches the same conclusion from its side, where its rule likewise improves on G&T-MWKR but not on EST until an objective that penalizes width is introduced [12].

The robust arm of Section 7.3 is the direct test of that reading: it is the one method in the comparison whose objective charges for the width, and it attains the lowest deviation of the pool, $\bar{\varepsilon} \times 10^3 = 5.23$ against 5.48 for the default arm, more faithful on 49 of the seventy instances ($p < 0.001$) and with the narrowest predictions (11.2% of the expected makespan against 11.8%). This is the direction the argument predicts, demonstrated at benchmark scale, and it comes at the price Section 7.3 measured: about one point of expected makespan at this λ. The result completes the chain begun there: the narrowing that Section 7.3 traced to sample selection is not confined to the predicted interval but carries through to execution, as schedules whose realized makespans stay closer to the value reported in advance. Whether that trade is worth making depends on the shop rather than on the model; the comparison above shows it is obtained only when the objective charges for the width.

7.5. Limitations

Five limitations bound the scope of these conclusions. First, the benchmark follows the symmetric ±15% generation scheme standard in the IJSP literature; asymmetric intervals, and distributions in which the uncertainty varies sharply from one operation to another, may separate the worst-case and expected-value criteria far more than they do here, and no policy was trained under them. The second benchmark changes the family of scheduling instances but not this scheme, so no result here speaks to a different uncertainty regime, and the finding that widths are inert as inputs is a statement about this objective on this construction rather than about interval scheduling at large. Second, the policy builds semiactive schedules, so the space it searches excludes the active schedules that the conflict-set restriction reaches and the insertion-based ones beyond them; the strongest constructive baseline of this paper exploits the first of those and the policy still leads it, but the ceiling is real. Third, the frontier of Section 7.3, though it runs on ten seeds per arm, is measured on the six validation instances, so where the width–makespan trade is produced is established on the class the policy was tuned on rather than on unseen instances. Fourth, the four training instances are always presented in the same order, so the variability the seeds measure is that of a fixed curriculum; whether a different order would move the arm was not tested. Finally, the comparison with the rule evolved by genetic programming shares the evaluation protocol end to end (the same instances, interval arithmetic, ranking criterion, inference budgets and execution realizations, run in a single harness) but not the training side: each learner was trained as its own study prescribes. Table 9 prices the two training bills in one currency, 4.4

against 16 hours for the thirty artifacts, but pricing is not parity, since neither family was granted the other's budget; the comparison of the two paradigms with training matched as well, which Xu et al. [43] call for, remains open.

8. Conclusions and Future Work

The three questions posed in the introduction now have answers.

Neither learned paradigm dominates the other: which of the two leads is decided by the inference budget, and they differ significantly at every budget evaluated. Given one constructive pass each the evolved rule leads, and once both methods sample the policy leads by a margin of the same size; the reading over the thirty artifacts of each family, which no selection touches, agrees with the one over the selected pair, and every difference is significant with the instance as the unit. What the network contributes over a compact formula is therefore its sampling distribution rather than its argmax, and that contribution is not free: the budget those figures equalize is the number of schedules constructed, while a policy costs some four times an evolved rule to train and about seven to eleven times as much per schedule (Table 9). The aggregate also hides a size effect, the policy leading near the class it trained on and trailing on the two largest. On the twelve classical instances, a family neither method was tuned for, it holds the lower figure at every budget. Margins under a point in either direction should finally be weighed against what the error metric does not register: the evolved rule carries less technical machinery, shorter development, training and inference times, a decision function that remains a readable expression and, on the evidence of the two largest classes, the more graceful transfer to instances far above the training size (Q1).

The paradigms differ in where they handle the uncertainty. For the policy the interval widths are inert as inputs: they sit at the floor of a permutation audit, and removing them from training costs nothing measurable on thirty instances no arm has seen (-0.02 points, the effect bounded within a third of a point either way). The design does not guarantee this: one shaping term of the reward reads a lower endpoint, so a channel into the weights exists and the inertness is a measured outcome. On the selection side the ranking criterion consults the lower endpoints only on exact ties of the upper, which decide seven of the three hundred measured pools and, where they do, choose among schedules with the same worst case. As training data the intervals are not inert, but only where the policy samples: fitting it to the crisp midpoints instead costs 0.43 points at the deployed budget ($p = 0.045$) and nothing at one constructive pass. An objective that charges for the width then separates the families. It narrows the policy's schedules through which sample is kept, the retrained weights being indistinguishable from the default ones under a common selection criterion and ranking the default policy's own samples by f_λ retracing the frontier without any retraining, where in the GP study the same objective acts through the evolved representation. What distinguishes the two is the stage at which robustness is bought rather than its rate, comparable on the two sides at roughly two points of RE per point of width. Bought deliberately, it reaches execution: the arm trained under that objective attains the lowest deviation between predicted and realized makespans of the whole comparison (Q2).

Both learners leave the hand-crafted rules behind by a margin that dwarfs the one between them, and neither reaches the per-instance metaheuristics, which pay for their lead with a

search on every new instance: the two learned constructive families occupy the same ground, between the baselines they dominate and the search algorithms they do not attempt to replace, and the choice between them is the budget question of Q1 rather than a difference in kind (Q3).

Three directions follow. The first concerns the representation, which the width-penalizing arm sharpens rather than settles: paying for the width narrows the schedules the policy *selects* without measurably changing the policy itself. Whether the gradient never reaches those features, or whether a semiactive constructive decision fixes the width before the policy has any say in it, separates two very different diagnoses, and a reward defined on the width alone would tell them apart. The second is deployment: a policy that schedules an instance in seconds is a candidate generator of initial solutions for population-based search, and the quality of its sampling distribution is the property such a use would exploit. The third turns the budget analysis inward. The network commits to a schedule at 19.5% while a few hundred of its own samples reach 14.2%, and the evolved rule shows the same separation, so the gap belongs to randomized constructive search rather than to learning as such. What differs is the remedy: the weights of a network can be moved onto its own best rollouts by retraining on them and iterating, the self-labeling strategy that Corsini et al. [6] demonstrate on the crisp JSP, where a formula would have to be evolved anew.

Funding

This work was supported by the Spanish Ministry of Science, Innovation and Universities (MCIN/AEI/10.13039/501100011033) under grant PID2022-141746OB-I00.

Declaration of competing interest

The author has no competing interests to declare that are relevant to the content of this article.

Data availability

The benchmark instances, the code, the benchmark harness, all experiment records (accepted and rejected), the final checkpoints and the figure-generation scripts are openly available in a Zenodo repository [11].

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the author used Claude (Anthropic) in order to assist with the drafting and editing of the manuscript and with the development and verification of the analysis scripts that recompute the reported numbers from the primary data. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the published article.

References

[1] Abdullah, S., Abdolrazzagh-Nezhad, M., 2014. Fuzzy job-shop scheduling problems: A review. Information Sciences 278, 380–407. doi:https://doi.org/10.1016/j.ins.2014.03.060.

[2] Bello, I., Pham, H., Le, Q.V., Norouzi, M., Bengio, S., 2016. Neural combinatorial optimization with reinforcement learning. ArXiv:1611.09940.

[3] Bertsimas, D., Sim, M., 2004. The price of robustness. Operations Research 52, 35–53. doi:https://doi.org/10.1287/opre.1030.0065.

[4] Branke, J., Nguyen, S., Pickardt, C.W., Zhang, M., 2016. Automated design of production scheduling heuristics: a review. IEEE Transactions on Evolutionary Computation 20, 110–124. doi:https://doi.org/10.1109/tevc.2015.2429314.

[5] Breiman, L., 2001. Random forests. Machine Learning 45, 5–32. doi:https://doi.org/10.1023/A:1010933404324.

[6] Corsini, A., Porrello, A., Calderara, S., Dell'Amico, M., 2024. Self-labeling the job shop scheduling problem, in: Advances in Neural Information Processing Systems (NeurIPS).

[7] Díaz, H., González-Rodríguez, I., Palacios, J.J., Díaz, I., Vela, C.R., 2020. A genetic approach to the job shop scheduling problem with interval uncertainty, in: Information Processing and Management of Uncertainty in Knowledge-Based Systems (IPMU), Springer. pp. 663–676. doi:https://doi.org/10.1007/978-3-030-50143-3_52.

[8] Díaz, H., Palacios, J.J., Díaz, I., Vela, C.R., González-Rodríguez, I., 2022. Robust schedules for tardiness optimization in job shop with interval uncertainty. Logic Journal of the IGPL 31, 240–254. doi:https://doi.org/10.1093/jigpal/jzac016.

[9] Díaz, H., Palacios, J.J., González-Rodríguez, I., Vela, C.R., 2023a. An elitist seasonal artificial bee colony algorithm for the interval job shop. Integrated Computer-Aided Engineering 30, 223–242. doi:https://doi.org/10.3233/ica-230705.

[10] Díaz, H., Palacios, J.J., González-Rodríguez, I., Vela, C.R., 2023b. Fast elitist ABC for makespan optimisation in interval JSP. Natural Computing 22, 645–657. doi:https://doi.org/10.1007/s11047-023-09953-2.

[11] Díaz Rodríguez, H., 2026a. Deep reinforcement learning for the interval job shop: checkpoints, records and code. Zenodo. doi:https://doi.org/10.5281/zenodo.21970431. [dataset]. https://doi.org/10.5281/zenodo.21970431.

[12] Díaz Rodríguez, H., 2026b. Genetic programming hyper-heuristics for the job shop scheduling problem with interval durations: robustness-aware interpretable rules that generalize across instance sizes. SSRN. doi:https://doi.org/10.2139/ssrn.7214286. preprint. https://doi.org/10.2139/ssrn.7214286.

[13] Gabel, T., Riedmiller, M., 2008. Adaptive reactive job-shop scheduling with reinforcement learning agents. International Journal of Information Technology and Intelligent Computing 24, 14–18.

[14] Garey, M.R., Johnson, D.S., Sethi, R., 1976. The complexity of flowshop and jobshop scheduling. Mathematics of Operations Research 1, 117–129. doi:https://doi.org/10.1287/moor.1.2.117.

[15] Giffler, B., Thompson, G.L., 1960. Algorithms for solving production-scheduling problems. Operations Research 8, 487–503. doi:https://doi.org/10.1287/opre.8.4.487.

[16] González-Rodríguez, I., Vela, C.R., Puente, J., 2010. A genetic solution based on lexicographical goal programming for a multiobjective job shop with uncertainty. Journal of Intelligent Manufacturing 21, 65–73. doi:https://doi.org/10.1007/s10845-008-0161-x.

[17] Grumbach, F., Müller, A., Reusch, P., Trojahn, S., 2024. Robust-stable scheduling in dynamic flow shops based on deep reinforcement learning. Journal of Intelligent Manufacturing 35, 667–686. doi:https://doi.org/10.1007/s10845-022-02069-x.

[18] Haupt, R., 1989. A survey of priority rule-based scheduling. OR Spektrum 11, 3–16. doi:https://doi.org/10.1007/bf01721162.

[19] Karunakaran, D., Mei, Y., Chen, G., Zhang, M., 2017. Dynamic job shop scheduling under uncertainty using genetic programming, in: Intelligent and Evolutionary Systems, Springer. pp. 195–210. doi:https://doi.org/10.1007/978-3-319-49049-6_14.

[20] Kasperski, A., Zieliński, P., 2016. Robust discrete optimization under discrete and interval uncertainty: A survey, in: Robustness Analysis in Decision Aiding, Optimization, and Analytics. Springer. doi:https://doi.org/10.1007/978-3-319-33121-8_6.

[21] Kayhan, B.M., Yildiz, G., 2023. Reinforcement learning applications to machine scheduling problems: a comprehensive literature review. Journal of Intelligent Manufacturing 34, 905–929. doi:https://doi.org/10.1007/s10845-021-01847-3.

[22] Kool, W., van Hoof, H., Welling, M., 2019. Attention, learn to solve routing problems!, in: International Conference on Learning Representations (ICLR).

[23] Kouvelis, P., Yu, G., 1997. Robust Discrete Optimization and Its Applications. Kluwer Academic Publishers, Dordrecht. doi:https://doi.org/10.1007/978-1-4757-2620-6.

[24] Kwon, Y.D., Choo, J., Kim, B., Yoon, I., Gwon, Y., Min, S., 2020. POMO: Policy optimization with multiple optima for reinforcement learning, in: Advances in Neural Information Processing Systems (NeurIPS).

[25] Lei, D., 2011. Population-based neighborhood search for job shop scheduling with interval processing time. Computers & Industrial Engineering 61, 1200–1208. doi:https://doi.org/10.1016/j.cie.2011.07.010.

[26] Leon, V.J., Wu, S.D., Storer, R.H., 1994. Robustness measures and robust scheduling for job shops. IIE Transactions 26, 32–43. doi:https://doi.org/10.1080/07408179408966626.

[27] López-Ibáñez, M., Dubois-Lacoste, J., Pérez Cáceres, L., Birattari, M., Stützle, T., 2016. The irace package: Iterated racing for automatic algorithm configuration. Operations Research Perspectives 3, 43–58. doi:https://doi.org/10.1016/j.orp.2016.09.002.

[28] Nazari, M., Oroojlooy, A., Snyder, L., Takáč, M., 2018. Reinforcement learning for solving the vehicle routing problem, in: Advances in Neural Information Processing Systems (NeurIPS).

[29] Nguyen, S., Mei, Y., Zhang, M., 2017. Genetic programming for production scheduling: a survey with a unified framework. Complex & Intelligent Systems 3, 41–66. doi:https://doi.org/10.1007/s40747-017-0036-x.

[30] Nguyen, S., Zhang, M., Johnston, M., Tan, K.C., 2013. A computational study of representations in genetic programming to evolve dispatching rules for the job shop scheduling problem. IEEE Transactions on Evolutionary Computation 17, 621–639. doi:https://doi.org/10.1109/TEVC.2012.2227326.

[31] Palacios, J.J., González-Rodríguez, I., Vela, C.R., Puente, J., 2015. Coevolutionary makespan optimisation through different ranking methods for the fuzzy flexible job shop. Fuzzy Sets and Systems 278, 81–97. doi:https://doi.org/10.1016/j.fss.2014.12.003.

[32] Panwalkar, S.S., Iskander, W., 1977. A survey of scheduling rules. Operations Research 25, 45–61. doi:https://doi.org/10.1287/opre.25.1.45.

[33] Park, J., Chun, J., Kim, S.H., Kim, Y., Park, J., 2021. Learning to schedule job-shop problems: representation and policy learning using graph neural network and reinforcement learning. International Journal of Production Research 59, 3360–3377. doi:https://doi.org/10.1080/00207543.2020.1870013.

[34] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O., 2017. Proximal policy optimization algorithms. ArXiv:1707.06347.

[35] Smit, I.G., Wu, Y., Troubil, P., Zhang, Y., Nuijten, W.P.M., 2024. Neural combinatorial optimization for stochastic flexible job shop scheduling problems. ArXiv:2412.14052.

[36] Song, W., Chen, X., Li, Q., Cao, Z., 2023. Flexible job-shop scheduling via graph neural network and deep reinforcement learning. IEEE Transactions on Industrial Informatics 19, 1600–1610. doi:https://doi.org/10.1109/TII.2022.3189725.

[37] Sotskov, Y.N., Matsveichuk, N.M., Hatsura, V.D., 2020. Schedule execution for two-machine job-shop to minimize makespan with uncertain processing times. Mathematics 8, 1314. doi:https://doi.org/10.3390/math8081314.

[38] Taillard, E., 1993. Benchmarks for basic scheduling problems. European Journal of Operational Research 64, 278–285. doi:https://doi.org/10.1016/0377-2217(93)90182-M.

[39] Tassel, P., Gebser, M., Schekotihin, K., 2021. A reinforcement learning environment for job-shop scheduling. ArXiv:2104.03760.

[40] Đurašević, M., Jakobović, D., 2018. A survey of dispatching rules for the dynamic unrelated machines environment. Expert Systems with Applications 113, 555–569. doi:https://doi.org/10.1016/j.eswa.2018.06.053.

[41] Vinyals, O., Fortunato, M., Jaitly, N., 2015. Pointer networks, in: Advances in Neural Information Processing Systems (NeurIPS).

[42] Wang, Y., Guo, T., Liu, Z., 2025. An expectation-maximization algorithm-based autoregressive model for the fuzzy job shop scheduling problem. ArXiv:2502.00018.

[43] Xu, M., Mei, Y., Zhang, F., Zhang, M., 2025. Learn to optimise for job shop scheduling: a survey with comparison between genetic programming and reinforcement learning. Artificial Intelligence Review 58, 160. doi:https://doi.org/10.1007/s10462-024-11059-9.

[44] Zaheer, M., Kottur, S., Ravanbakhsh, S., Póczos, B., Salakhutdinov, R., Smola, A., 2017. Deep sets, in: Advances in Neural Information Processing Systems (NeurIPS).

[45] Zhang, C., Cao, Z., Song, W., Wu, Y., Zhang, J., 2024. Deep reinforcement learning guided improvement heuristic for job shop scheduling, in: International Conference on Learning Representations (ICLR).

[46] Zhang, C., Song, W., Cao, Z., Zhang, J., Tan, P.S., Xu, C., 2020. Learning to dispatch for job shop scheduling via deep reinforcement learning, in: Advances in Neural Information Processing Systems (NeurIPS).

[47] Zheng, P., Xiao, S., Zhang, P., Lv, Y., 2024. A two-individual-based evolutionary algorithm for flexible assembly job shop scheduling problem with uncertain interval processing times. Applied Sciences 14, 10304. doi:https://doi.org/10.3390/app142210304.